

Так, це краще відповідає нашому розділенню відповідальності.

У Region :

- `*armies[]` — system-computed зв'язок: infrastructure надає список присутніх Army.
- `camp_players[]` — звичайна computed-характеристика: Region на основі `*armies[]` визначає унікальний список Player, чиї війська зараз перебувають у Camp.

Тоді `RegionControl`, маючи прямі `player` і `region`, для `can_progress` може перевірити:

- `player ∈ region.camp_players[]` — свої війська присутні;
- чи є в `region.camp_players[]` хтось, крім `player` — присутні чужі війська.

Тобто окремі `has_player_armies` / `has_foreign_armies` у `RegionControl` справді не потрібні.

І це добре формалізує межу **system-computed**: infrastructure не повинна виконувати game-domain агрегації на кшталт «чи є тут війська саме цього Player». Вона лише надає те, що модель сама принципово не може отримати зі свого state — наприклад reverse relation `Region -> *armies[]`. Подальша інтерпретація цього зв'язку — вже game logic і звичайні computed-характеристики.

Sunday 12:31 AM

Важливий нюанс: `camp_players []` тут не зв'язок з Player (ме не можемо звертатися `camp_players[$i]->some_param`) а id гравців для порівняння

Так. Це важлива різниця.

`camp_player_ids[]` я б навіть назвав саме так, щоб семантика була однозначною: це **список scalar ID**, а не список посилань на `Player`.

Тобто:

- `*armies[]` — system-computed, список посилань на `Army` ;
- `camp_player_ids[]` — computed, список ID гравців, чиї Army перебувають у Camp у цій Region.

Тоді `RegionControl.can_progress` просто порівнює `player.id` із `region.camp_player_ids[]` і перевіряє наявність інших ID.

Правило `[]` при цьому означає лише «характеристика є колекцією», а не обов'язково «колекція зв'язків». Для посилань/зв'язків це визначається типом самої характеристики.

Можливо, Region варто мати список армій, які в ній знаходяться, не як system-computed, а як пряма характеристика (армії будуть напряму додаватися туди при вході і видалятися при виході).

Так, можна, але я б тут залишив `Region.armies[]` **system-computed**, а `Army.current_region` — прямою.

Причина не стільки в БД, скільки в появі двох джерел істини:

```
Army.current_region = R
і одночасно
R.armies[] містить Army .
```

Якщо `armies[]` зробити прямою, кожен вхід/вихід Арму мусить атомарно підтримувати обидві сторони. Те саме доведеться враховувати при battle, split/merge, rollback тощо. Розсинхронізація стає можливою.

З поточною схемою:

```
Army.current_region — direct, єдине джерело істини;
Region.*armies[] — system-computed reverse relation за current_region ;
Region.camp_player_ids[] — звичайна computed на основі *armies[] .
```

Це якраз той випадок, для якого ми й ввели system-computed: зворотний зв'язок існує об'єктивно, але його немає сенсу дублювати в state.

Виняток був би виправданий, якби `Region.armies[]` мала **самостійну ігрову семантику**, яку не можна однозначно відновити з `Army.current_region` . Але поки я такої потреби не бачу.

Добре, залишаєм computed. Внеси правки в опис характеристик моделей, про static id ще подумаєм, як його сформулювати і додати в опис архітектури

Worked for 2m

Вніс у `MODELS.md` :

- `Region.*armies[]` залишив system-computed.
- Додав `Region.camp_player_ids[]` як звичайну computed-характеристику зі scalar ID.
- Додав `Region.valid_connected_neighbor_player_ids[]` для тієї ж проблеми з сусідніми connected regions.

- Прибрав із RegionControl штучні *player_camp_armies[] , *other_players_camp_armies[] , *adjacent_regions[] .
- RegionControl.can_progress тепер використовує характеристики region .
- Аналогічно прибрав *foreign_camp_armies[] із CastleFounding ; він використовує region.camp_player_ids[] .
- Архітектурний файл не чіпав.

Один commit у main : e7a1f76b60b4598ed6eb8190cae42f6cbfc642fb .

Sunday 2:11 PM

Пропозиція: замість RegionControl зробити модель CampInRegion - вона буде представляти наявність армій певного гравця в кемпі в області, не тільки нейтральній/окупованій але й власній. Вона бере всі функції RegionControl , а також може використовуватися для перегруповання армій і їх відправку в похід

Так, мені ця модель здається логічнішою за RegionControl .

RegionControl фактично вже представляв не сам «контроль області», а стан пари Player + Region , пов'язаний із присутністю його військ. CampInRegion робить цю семантику явною: **один instance = один Player, який має війська в Camp конкретної Region.**

Це дає кілька переваг.

- CampInRegion існує для будь-якої області — своєї, чужої, нейтральної. Не треба створювати спеціальну сутність лише через можливість Annexation.
- Сам факт існування active CampInRegion уже означає, що Player має війська в Camp. Тобто частина нинішніх camp_player_ids[] /перевірок може стати непотрібною.
- control_progress , can_progress , is_ready природно переїжджають із RegionControl сюди. Для власної Region вони просто не використовуються.
- Перегрупування справді логічно прив'язане саме до Camp, а не до окремої Army: у Camp можуть бути кілька Unit/Army одного Player, з яких гравець формує новий склад.
- Відправлення в похід теж природно починати з CampInRegion : гравець вибирає війська зі свого Camp, після чого створюється/формується Army та Movement .

Але тут виникає важливе питання lifecycle. Я б визначив його так: CampInRegion створюється при появі першої Army/Unit Player у Camp Region і переходить у неактивний/terminal стан, коли останні

його війська залишають Camp. **Не видаляти**, як і інші persistent process/interactions, щоб зберігалася історія і rollback.

Тоді Region може мати system-computed:

*camps[] — active CampInRegion цієї Region.

A CampInRegion :

player — direct

region — direct

*armies[] — system-computed Army цього Player, які перебувають у цьому Camp.

Це також вирішує попередню проблему значно чистіше. Замість Region.camp_player_ids[] маємо справжню domain-сутність. Інша модель, якій треба знати про присутність гравців у Camp, працює з Region.*camps[], не лізучи Region -> Army -> Player .

Єдине, що я б поки не переносив автоматично — Army.regrouping_progress . Треба окремо визначити, що саме означає regrouping за правилами. Якщо це процес **усього Camp**, тоді regrouping_progress дійсно має переїхати в CampInRegion . Якщо це стан конкретної Army після певної дії/бою — лишається в Army .

У цілому я за заміну RegionControl → CampInRegion . Вона не просто перейменовує модель, а робить її більш фундаментальною і прибирає кілька штучних характеристик, які ми щойно були змушені вводити.

обчислювальний Region.camp_player_ids[] все ще потрібен.
regrouping_progress залишається в Army. Army в camp має пряме посилання на CampInRegion , додається туди прямо при вході в кемп і видаляється при виході, інакше ми не зберев все до купи

Так, тоді структура виходить чистіша.

CampInRegion — persistent representation конкретного Camp пари (Player, Region) .

У Army :

- camp — **пряма** характеристика-посилання на CampInRegion , якщо Army зараз у Camp;
- при вході в Camp посилання встановлюється;
- при виході — очищається;

- `regrouping_progress` залишається в `Army` .

У `CampInRegion` :

- `player` — direct;
- `region` — direct;
- `*armies[]` — system-computed reverse relation за `Army.camp` ;
- характеристики колишнього `RegionControl` : `control_progress` , `can_progress` , `control_progress_rate` , `required_control_progress` , `is_ready` .

У `Region` :

- `*armies[]` — як і раніше всі `Army`, фізично присутні в `Region`;
- `camp_player_ids[]` — computed список ID `Player`, які мають війська саме в `Camp`. Його можна обчислювати через `*armies[]` та їх прямий `camp` (або пізніше вирішити, чи зручніше через `camps`).

Це також дає важливу відмінність: `Army.current_region` відповідає на питання «**де фізично Army?**», а `Army.camp` — «**до якого локального Camp вона зараз належить?**». Тому це не дублювання одного й того самого зв'язку.

І погоджуюсь щодо «зберегти все до купи»: `CampInRegion.*armies[]` як reverse relation від прямого `Army.camp` значно надійніше, ніж щоразу реконструювати належність до конкретного `Camp` із кількох ознак `Army`.

Я б тільки не робив `CampInRegion` terminal при тимчасовому зникненні останньої `Army`. Якщо це постійне представлення пари (`Player`, `Region`) , яке ще й накопичує `control_progress` , логічніше зберігати той самий instance, поки правила прямо не вимагають скинути його стан. Інакше lifecycle `Camp` і lifecycle `control progress` почнуть конфліктувати.

`is_ready` потрібно перейменувати на щось типу `is_ready_for_annexation` (напиши правильно). `control_progress` не накопичується, якщо немає армій в кемпі. Якщо всі армії пішли - модель можна видаляти (не фізично)

Так.

Назва: `is_ready_for_annexation` — граматично й семантично нормально.

Тоді lifecycle `CampInRegion` :

- створюється при появі першої `Army` цього `Player` у `Camp` цієї `Region`;
- поки є хоча б одна `Army` — модель активна;

- `control_progress` накопичується лише за виконання інших умов `Annexation`; відсутність `Army` означає, що процес не може продовжуватися;
- коли остання `Army` залишає `Camp`, `CampInRegion` логічно видаляється / деактивується, але запис фізично залишається для `history/rollback`;
- при наступній появі військ створюється новий `instance CampInRegion`, а не відновлюється старий.

Тобто `CampInRegion` тепер добре представляє саме **один безперервний епізод присутності `Player` у `Camp Region`**, а не постійне відношення `Player` ↔ `Region`. Це важливе уточнення.

внеси зміни в MODELS

Thinking

Вніс у `MODELS.md`:

- `RegionControl` замінено на `CampInRegion`;
- описаний `lifecycle CampInRegion`;
- перенесені `control_progress`, `can_progress`, `Annexation` logic і `triggers`;
- `is_ready` → `is_ready_for_annexation`;
- в `Army` додано прямий `camp`;
- `CampInRegion.*armies[]` — reverse system-computed;
- `Region.camp_player_ids[]` залишено;
- `regrouping_progress` залишається в `Army`;
- `Movement.camp_reached` враховує створення/підключення до `CampInRegion`.

Один `commit` у `main`: `08f7896094152206dc5b9a95cf87476e9fb41533`.

1. по списку тригерів додай в моделі методи `check_trigger` та `on_trigger`
2. Проаналізуй наявні юзєр-методи і методи тригерів в моделях. Які додаткові методи вони будуть використовувати (в тому числі в пов'язаних моделях)? Додай ці нові методи, в кожному методі опиши, які інші методи він використовує.
3. Проаналізуй всі наявні методи, опиши також, які характеристики ці методи використовують. Якщо якихось характеристик не вистачає - додавай.

По результату має буде повний опис характеристик, тригерів і методів моделей, що описують всю бек-енд ігрову логіку.

Worked for 10m 14s

Оновив docs/MODELS.md одним комітом у main .

Додав:

- `check_trigger_*`() і `on_trigger_*`() для всіх наявних triggers;
- внутрішні domain methods, які використовуються user/trigger methods;
- для кожного method — які характеристики він використовує і які інші methods викликає;
- відсутні характеристики, потрібні для цієї логіки;
- додаткові reverse/system-computed relationships, де вони потрібні;
- повніші flow для Movement, Camp, Annexation, Combat, Recruitment, upgrades, Founding, Neutral Defense recovery, connectivity, Knight replacement.

Також прибрав кілька слабких місць попередньої структури, наприклад `KnightReplacement` тепер визначає свою чергу через computed характеристику `Castle`, а не через штучний прямий список інших replacement-процесів.

Commit: 3b1b94806745e7ec1b017b64bcb44b384d4217c4 .

Тепер проаналізуй файл з правилами і ще раз файл з описом моделей. Чи дійсно тригери й методи покривають всю логіку.

Worked for 2m 17s

якщо щось не виходить описати прийнятою архітектурою - кажи, будем обговорювати

Ні. Після повторної звірки `GAME_RULES` з поточним `MODELS.md` видно кілька реальних прогалин і кілька місць, де опис моделей навіть суперечить правилам.

Найсуттєвіші:

1. Немає user-action для атаки іншого гравця в Neutral Region. Правила дозволяють гравцю з Camp атакувати іншого гравця, чиї війська теж у Camp, причому Defender — всі його Army в цьому Camp. У моделях є лише `user_attack_neutral_defense()` . Потрібен окремий метод на кшталт `CampInRegion.user_attack_player(...)` або `CombatSituation.user_attack_player_camp(...)` .
2. Pending `CombatSituation` недостатньо пов'язана з Region/Camp. За правилами Army, яка заходить у власну атаковану Region до початку battle і має її кінцевою точкою, автоматично приєднується до defense. Поточному `Region.resolve_arrival()` нема звідки надійно дізнатися

про вже існуючу pending CombatSituation. Потрібен природний reverse relationship, найімовірніше `Region.*combat_situations[]`, і метод пошуку relevant pending combat.

3. `camp_player_ids[]` зараз недостатньо точно визначена. Regrouping не рахується присутністю для Annexation/Founding, але Regrouping Army фізично лишається в Camp і може оборонятися. Отже треба розділити щонайменше:
 - o eligible Camp presence для Annexation/Founding;
 - o defensive/local military presence для combat.
 Один список `camp_player_ids[]` не може коректно обслуговувати обидва випадки без уточнення семантики.
4. Через це `Region.resolve_arrival()` не повинен визначати defenders лише через `camp_player_ids[]`. Regrouping Army повинна вступити в defense, хоча для Annexation вона не рахується.
5. Є серйозна суперечність між `Army.camp` і правилом Castle/Camp. У правилах окремі Unit всередині однієї Army можуть бути частково в Castle, частково в Camp тієї самої Castle Region. Але `Army.camp` є одним прямим посиланням для всієї Army. Це не описує такий стан. Тут поточна архітектура справді не складається без додаткового рішення. Ймовірно, Camp/Castle placement має належати Unit/Knight, а не лише Army, але це треба окремо обговорити.
6. Та сама проблема проявляється при `CampInRegion.*armies[]`: якщо Army частково в Castle, частково в Camp, не можна однозначно сказати, що вся Army належить Camp. Це найбільше структурне питання з усього списку.
7. Combat parameters зараз можуть lock-атися занадто рано. `user_attack_neutral_defense()` викликає `lock_combat_parameters()` при створенні CombatSituation. Але правила кажуть, що thresholds/classification фіксуються **в момент початку combat**, а не при його ініціації. До battle start thresholds можуть змінитися. `lock_combat_parameters()` має виконуватися тільки в `on_trigger_battle_start()` або безпосередньо перед resolve.
8. Retreat flow неповний. `Army.retreat_to()` описує перехід у сусідню Region і майбутній Regrouping, але за правилами між ними є реальний локальний рух до Camp тривалістю D. У нас Movement вже має retreat-local, але `retreat_to()` явно не створює/не запускає такий Movement. Треба зв'язати ці два flow.
9. Retreat selection описаний не повністю. `get_legal_retreat_regions()` знаходить допустимі Region, але правила ще задають пріоритети й випадковий вибір при рівності. Потрібен окремий `select_retreat_region()` або розширення цього method. Також `user_set_pre_battle_retreat_decision(army, destination)` виглядає підозріло: у GAME_RULES користувач фіксує рішення Retreat, а destination далі визначається правилами. Я б не дозволяв користувачу передавати destination, якщо ми окремо цього не погоджували.

10. `CastleFounding.founder_valid` потребує точного факту «Knight перебуває у звичайному Camp». Зараз через `founder_knight` можна прочитати Knight, але щоб дізнатися його Camp через Knight -> Army -> camp, виникає заборонений ланцюг relationship. Треба підняти в Knight computed характеристику на кшталт `camp / is_in_camp / camp_state`, яку `CastleFounding` може читати безпосередньо.
11. City Raid описаний не повністю. Правила дозволяють raid-specific resistance Neutral Defense, причому часткові втрати Neutral Defense не зберігаються. Поточний `user_raid_city()` лише загально каже «створити CombatSituation». Треба чіткий raid combat type і окрема result logic, щоб raid не викликав `Region.destroy_neutral_defense()` як звичайна атака для Annexation.
12. У City є ще одна потенційна прогалина: правила дозволяють тимчасове зменшення Wealth та/ або Coin income після Raid. Якщо це справді тимчасовий debuff, а не просто зміна Wealth, зараз для нього немає characteristic/process/trigger recovery. Якщо в V1 ми вирішимо реалізовувати raid лише через зміну Wealth — тоді окремого механізму не потрібно. Це треба уточнити.
13. Смерть Knight описана недостатньо. `Knight.die()` створює replacement, але не прибирає померлого Knight із Army. Якщо помирає не Commander, `Army.*knights[]` може продовжити бачити dead Knight через `Knight.army`. Потрібен контрольований remove/detach. Якщо помирає Commander, dissolve частково це вирішує, але загальний lifecycle membership все одно треба описати.
14. Occupation restore має race/order ризик усередині однієї GameEvent.
`CampInRegion.on_army_left()` може викликати `Region.restore_owner_control()`. Якщо старий occupier виходить у той самий event, коли новий переможець займає Region, restore не повинен стерти вже нового occupier. Краще `restore_owner_control(expected_occupier)` і очищення тільки якщо current occupier усе ще той самий.
15. Економіка в цілому покрита, але Food consumption у Region треба деталізувати.
`Region.food_flow_to_castle` має враховувати саме Camp troops, а не всі `*armies[]`, бо Movement Food локально не споживає. Regrouping фізично присутній і, за правилами, ймовірно лишається Camp consumption. Тут потрібна окрема аргументована computed характеристика, а не використання всіх Army без фільтра.

Тобто основний каркас уже є, але я б поки не називав `MODELS.md` повним описом усієї backend game logic.

Найперше я б розібрав пункт 5 — **Army / Unit / CampInRegion при ситуації, коли Unit однієї Army частково в Castle, частково в Camp**. Це не просто відсутній method; тут поточне представлення стану не може однозначно описати правило гри. Після цього вже можна механічно закрити решту пунктів.

Стан солдатів Castle/Camp впливає тільки на вартість утримання солдатів і місце в казармі. Від визначається в Knight. Для всіх логік переміщення і взаємодії військ армія лицаря вважається в CampInRegion. Тобто:

Army, в яку входить Knight - в стані Camp, має активний CampInRegion.

Knight при цьому має location_state, який може бути Castle / Camp, який ніяк не впливає на Army. location_state може бути Castle тільки тоді, коли армія в Camp і Camp.region = рідний замок лицаря

Тоді пункт 5–6 з мого попереднього списку знімається — ця модель узгоджується.

Фіксуємо так:

- Army має стан Camp і пряме посилання camp -> CampInRegion .
- Усі військові взаємодії, переміщення, combat, Annexation/Founding presence тощо дивляться на Army і CampInRegion .
- Knight.location_state — окремий локальний стан лише для economy/housing:
 - Castle
 - Camp
- Він впливає тільки на:
 - Barracks capacity;
 - upkeep Soldier;
 - можливість Castle reserve <-> Knight .
- На Army.camp , combat presence, movement presence та іншу польову логіку location_state не впливає.
- location_state = Castle допустимий лише якщо:
 - Army.state == Camp ;
 - Army.camp != null ;
 - Army.camp.region == Knight.castle.region .

Тобто навіть якщо частина Knight однієї Army фізично «в казармі», а частина «в таборі», вся Army з точки зору карти й interaction однаково знаходиться в одному CampInRegion .

Це навіть спрощує модель: CampInRegion.*armies[] лишається коректним reverse relation по Army.camp , а Knight.location_state не треба враховувати в Camp presence.

Єдине, що я б додав у `MODELS.md` пізніше — `computed` на `Knight` на кшталт `is_housed_in_castle` або просто чітко описав `constraint location_state == Castle`, щоб `barracks_used` і `upkeep` не залежали від розмитих трактувань.

Після цього найбільш суттєві відкриті прогалини з мого списку — це вже не представлення `Camp`, а:

- атака `Player vs Player` у `Neutral Camp`;
- `pending CombatSituation/reinforcements`;
- `retreat flow` через `Movement` ;
- `timing lock combat parameters`;
- `Founding founder presence`;
- `raid-specific Neutral Defense logic`;
- `Knight death/detach`;
- `precise eligible Camp presence vs defensive presence`.

є в списку щось, що не вирішується /складно вирішується
поточною архітектурою?

Після уточнення про `Knight.location_state` — ні, я не бачу в тому списку нічого, що вимагало б зміни базової архітектури.

Усе інше вкладається в уже прийняті механізми: `direct/system-computed/computed/dynamic characteristics, relationships, methods, triggers` та `event actions`.

Найбільш «незручні» місця, але вони все одно нормально вирішуються:

- **Pending CombatSituation + підключення нових defenders.** Потрібен `system-computed reverse relation` на кшталт `Region.*combat_situations[]` і метод, який при `arrival` знаходить актуальну ситуацію. Це стандартний для нас випадок.
- **Eligible presence vs combat presence.** Не архітектурна проблема. `CampInRegion.*armies[]` дає `Army`, а `computed`-характеристики `CampInRegion` можуть агрегувати потрібні стани: хто рахується для `Annexation`, хто для `defense` тощо.
- **CastleFounding і стан founder.** Заборонений ланцюг `CastleFounding -> Knight -> Army -> CampInRegion` просто розриваємо за нашим правилом: `Knight` отримує потрібну `computed`-характеристику, наприклад `current_camp`, яка вже агрегує інформацію нижчого рівня. `CastleFounding` читає її.
- **Retreat.** `CombatSituation -> Army.retreat...() -> Movement` — звичайний ланцюг `domain methods`. Тут просто бракує опису.

- **Knight death / Army membership, Occupation race, combat parameter locking, raid-specific resolution** — це все питання правильних methods/event actions, а не архітектури.

Є лише один момент, який я б тримав під наглядом: наше правило «**computed characteristic не ходить через ланцюг relationships**». Воно породжуватиме проміжні computed-характеристики на моделях, як у випадку `CastleFounding -> Knight -> Army -> Camp`. Але поки це виглядає не проблемою, а корисною інкапсуляцією: кожна модель надає наступному рівню саме ту інформацію, яка йому потрібна.

Тобто зараз я б **не змінював архітектуру**. Можна послідовно закривати знайдені прогалини в `MODELS.md`.

Sunday 5:46 PM

деталізуєм споживання їжі:

армія в кемпі на своїй території:

- споживає їжу, яка виробляється в цій області (споживання армією віднімається від виробництва в області до застосування коефіцієнту відстані)
- якщо виробництва недостатньо - споживання включається в баланс замку з урахуванням коефіцієнту відстані (їжа постачається з замку). Замок, до якого приєднана область, а не рідний замок лицарів.
- якщо запас нульовий і баланс ≤ 0 - нестача компенсується монетами

армія в кемпі на чужій або нейтральній території:

- споживає їжу, яка виробляється в цій області
- якщо виробництва недостатньо - нестача компенсується монетами

В константах гри задається фіксований курс монет до їжі. Армія не переходить повністю на утримання монетами, монети компенсуються саме нестачу їжі. Тобто, якщо армія споживає 100 одиниць, а область виробляє 90 - монети рахується по курсу за 10 одиниць їжі.

Якщо в одній області є декілька армій, в тому числі різних гравців, і виробництва їжі на всіх недостатньо, нестача

розподіляється пропорційно, ніякий гравець або армія не мають пріоритету.

Армія в тринзиті повністю споживає монети замість їжі по тому самому курсу. Можна було б для транзити по своїй території залишити споживання з поточної області, але не хочу ускладнювати правила - що думаєш?

Солдати мають окреме постійне споживання монет, незалежне від їжі. Компенсація нестачі їжі додається до звичайного споживання.

Я б для Transit залишив саме повне споживання через Coins незалежно від того, своя це територія чи ні.

Це дає чітке правило станів:

- Camp → спочатку локальне виробництво Food;
- Camp у своїй Region → нестача додатково може покриватися Food із Castle цієї Region;
- решта нестачі → Coins;
- Transit → усе food-equivalent consumption одразу переводиться в Coins;
- базове Soldier coin upkeep при цьому існує завжди і просто додається зверху.

Варіант із використанням Food поточної Region під час Transit створює багато додаткових питань: коли саме Army перестає споживати з попередньої Region, чи споживає під час переходу між Region, як розподіляти production між Camp і transit Army, як поводитися на своїй/чужій території тощо. Gameplay-виграш від цього невеликий.

Є один важливий момент у твоєму правилі для власної Region, який треба точно зафіксувати математично. Наприклад, Army потребують сумарно 100 Food, Region виробляє 90. Локальний дефіцит = 10. Якщо distance coefficient для доставки між Castle і Region, умовно, 0.5, то для компенсації цих 10 Army із Castle має витратитися **20 Food**, а не 10. Тобто коефіцієнт застосовується до доставки з Castle так само, як до потоку виробленої Food у Castle, тільки у зворотному сенсі. Якщо саме це ти маєш на увазі — схема однозначна.

Також пропорційний розподіл я б застосовував поетапно. Якщо в Region дві Army споживають 60 і 40, а локального production 50, вони отримують відповідно 30 і 20. Їх дефіцити — 30 і 20. Далі для своєї Region цей дефіцит намагається покрити Castle; якщо доступного Food Castle також недостатньо, його знову треба розподілити пропорційно між усіма відповідними Army, а залишок

кожної Army конвертується в Coins. Це забезпечує правило «жодна Army/Player не має пріоритету» на кожному обмеженому джерелі.

Для backend це теж добре вкладається в поточну архітектуру. Я б не рахував food upkeep окремо всередині кожної Army — через пропорційний розподіл результат Army залежить від **усіх Army в Region**. Природне місце розрахунку — Region : вона знає `*armies[]`, `production` і свій `castle`, обчислює розподіл локальної Food та потребу в постачанні з Castle. `Castle.food_balance` уже використовує результати своїх `regions[]`, а `Player.coin_balance` отримує сумарну food-deficit compensation своїх військ плюс звичайний Soldier coin upkeep.

Тут, щоправда, при деталізації характеристик треба уважно розірвати залежності, щоб не отримати цикл Region food allocation → Castle food_balance → Region allocation. Це вирішується, але перед внесенням у `MODELS.md` я б спочатку окремо розклав характеристики та напрямки залежностей.

Все вірно. Недостача виробництва в області компенсується доставкою з замку в урахуванням коефіцієнту відстані.

Так. Тоді фіксуємо саме так: якщо Region виробляє P , а Camp Army сумарно потребують C , то спочатку локально покривається $\min(P, C)$. Якщо $C > P$, дефіцит $D = C - P$.

Для своєї Region цей D покривається доставкою з Castle, до якого приєднана Region. Якщо коефіцієнт доставки $k < 1$, для доставки D одиниць до Army із запасу Castle потрібно витратити D / k Food. Якщо доступної Food недостатньо, фактично доставлена кількість розподіляється між Army пропорційно їх дефіцитам, а залишок дефіциту кожної Army компенсується Coins.

Для чужої/нейтральної Region етап Castle supply відсутній: після локального виробництва весь залишковий дефіцит одразу переводиться в Coins.

У Transit: `food_need × coin_per_food` повністю додається до coin consumption незалежно від території.

І окремий постійний Soldier coin upkeep завжди додається до цього значення.

Цю схему можна вже вважати зафіксованою для подальшого опису характеристик Region, Castle, Army і Player.

В області, що налажить гравцю, не може бути одночасно армії різних гравців, тому розподіл нестачі між арміями тут рахувати не треба.

В нейтральній території треба рахувати розподіл нестачі між гравцями а не конкретними арміями. Споживання їжі однакове для всіх типів солдатів, лицар теж рахується як одна одиниця.

Так, це спрощує розрахунок.

Для Owned Region:

- у Camp можуть бути війська тільки власника Region;
- локальне виробництво Food спочатку покриває їх сумарне споживання;
- дефіцит покривається доставкою з Castle цієї Region з урахуванням distance coefficient;
- залишковий дефіцит компенсується Coins власника;
- пропорційний розподіл між Army не потрібен, бо для економіки важливий сумарний дефіцит одного Player.

Для Neutral Region:

- визначаємо потребу кожного Player за кількістю його військ у Camp;
- кожен Soldier = 1 food-consumption unit;
- кожен Knight = 1 food-consumption unit;
- тип Soldier на Food consumption не впливає;
- локальне виробництво розподіляється між Player пропорційно кількості їх food-consumption units;
- дефіцит кожного Player окремо переводиться в Coins за фіксованим курсом.

Наприклад: у Player A 80 Soldier + 2 Knight = 82 units, у B — 17 Soldier + 1 Knight = 18 units. Загальна потреба 100. Якщо Region дає 60 Food, A отримує 49.2, B — 10.8; відповідно coin compensation рахується за дефіцит 32.8 і 7.2.

Це також означає, що для економічної логіки Region нам корисні не Army як такі, а агрегована характеристика на кшталт `camp_food_consumption_by_player`, де ключ — Player ID, значення — сумарна потреба його Knight + Soldier у Camp. Вона природно обчислюється Region на основі `*armies[]`.

За споживання їжі в Campу може відповідати `CampInRegion` а не `Army`. Region може мати характеристику загальна кількість споживання їжі по всім гравцям (для нейтральної), і булеву характеристику, чи є дефіцит. У випадку дефіцита кожен `CampInRegion` буде сам рахувати свою частку. Яюсь так.

Так, це виглядає чистіше. Я б розподілив відповідальність саме так.

`CampInRegion` відповідає за споживання конкретного `Player` у конкретній `Region`. Він має `computed food_consumption` — сумарне споживання всіх `Knight + Soldier` у `*armies[]`. Тобто це вже агрегат по гравцю, і окремі `Army` в подальшому `food`-розрахунку не беруть участі.

`Region` агрегує всі `Camp`. Їй потрібні, наприклад:

- `*camps[]` — active `CampInRegion` ;
- `camp_food_consumption` — сумарний `food_consumption` усіх `Camp`;
- `has_food_deficit` — `camp_food_consumption > food_production` .

Тоді `CampInRegion` може рахувати свою частку локального `Food`. Якщо дефіциту немає — його потреба повністю покривається `Region`. Якщо є — частка локальної `Food` пропорційна його `food_consumption` :

`region.food_production × camp.food_consumption / region.camp_food_consumption` .

Це не створює поганого ланцюга: `CampInRegion` -> `Region` читає вже готові агреговані характеристики `Region`, а `Region` отримує `*camps[]` і читає безпосередньо `CampInRegion.food_consumption` .

Для `Owned Region` після цього конкретний `CampInRegion` може визначити `food_deficit_after_local` . Далі цей дефіцит постачається з `region.castle` з `distance coefficient`, а непокритий залишок перетворюється на `coin consumption`.

Але тут є один нюанс нашого правила про ланцюги: `CampInRegion` -> `Region` -> `Castle` -> `food` нам робити не можна. Тому інформацію, потрібну `Camp` для `castle supply`, треба підняти в `Region` . Наприклад, `Region` може мати `computed available_castle_food_supply` або іншу характеристику, яка вже враховує її `Castle` і `distance coefficient`.

І тут треба бути обережним із кількома `Owned Regions` одного `Castle`: вони одночасно претендують на один запас `Food Castle`. Просте `Region.available_castle_food_supply` може створити ситуацію, коли кожна `Region` вважає доступним увесь запас `Castle`. Це вже місце, де розподіл дефіцитного `Castle Food` між його `Regions` треба визначити окремим правилом.

Тобто локальний розподіл `Food` через `Region` + `CampInRegion` виходить дуже добре. Наступне питання, яке нам треба визначити: **що відбувається, якщо `Food` у `Castle` недостатньо для одночасного покриття дефіциту кількох його `Regions`.**

Sunday 9:03 PM

Проблема: якщо гравець тримає велике армію в своїй області, далекій від замку, через великі витрати на транспортування, може бути вигідніше перейти на утримання за монети, ніж постачати їжу з замку. Але ми такої можливості не даємо. Виходить, тримати армію в своїй далекій області може бути дорожче, ніж в далекій нейтральній. Якщо немає ідей, як це пофіксити без ускладнення - доведеться відмовитись від постачання їжі з замку

Так, проблема реальна: при коефіцієнті доставки k одна одиниця їжі для армії коштує замку $1/k$ Food, тоді як на нейтральній території ця сама одиниця дефіциту просто коштує фіксовану кількість Coins. За достатньо малого k власна Region стає економічно гіршою.

Є досить простий спосіб це виправити без вибору режиму гравцем: **система автоматично використовує дешевший спосіб утримання.**

Для дефіциту локальної їжі в Owned Region:

- якщо доставка 1 Food із Castle з урахуванням коефіцієнта вигідніша за компенсацію 1 Food монетами — використовується Castle Food;
- якщо дорожча — дефіцит одразу компенсується Coins.

Але тут виникає неприємність: Food і Coins — різні ресурси, тому поняття «дорожче» не існує без додаткового курсу **Food y Castle → Coins**. Наш `coin_per_food` визначає вартість утримання армії монетами, але сам по собі не визначає економічну цінність запасу Food. Вводити ще одну оцінку ресурсу лише заради цього я б не став.

Можна зробити ще простіше: обмежити втрати при транспортуванні. Наприклад, для постачання армії використовувати $\max(\text{distance_coefficient}, 0.5)$. Тоді далека Army максимум витрачає 2 Food із Castle за 1 Food споживання. Але це вже досить штучне спеціальне правило.

Тому з трьох варіантів:

1. вибір Food/Coins гравцем — зайва мікромеханіка;
2. автоматичний вибір — потребує економічної оцінки Food;
3. спеціальний `minimum supply coefficient` — простий, але штучний.

Я б вибрав четвертий і найчистіший: **відмовитися від Castle → Army Food supply.**

Тоді правило стає дуже простим і симетричним:

CampInRegion спочатку споживає свою частку локального виробництва Region → будь-який дефіцит компенсується Coins.

Неважливо, Owned це Region чи Neutral. Відстань до Castle впливає на те, скільки надлишкової їжі Region доставляє в Castle, але не на утримання військ у Region.

На мій погляд, це навіть краще як game rule: війська харчуються з місцевого постачання, а якщо його бракує — Player платить за зовнішнє постачання. І ми повністю прибираємо дивний випадок, коли завоювання Region робить утримання тієї самої Army дорожчим.

добре. Відмовимся. Але з виключенням - армії що стоять в області з замком - споживають їжу з балансу замку

Так. Тоді правило стає послідовним:

- Army в CampInRegion спочатку споживає локальне виробництво Food своєї Region.
- Якщо локального виробництва недостатньо, дефіцит компенсується Coins.
- Немає постачання Food із Castle у приєднані Region незалежно від відстані.
- Виняток — **Castle Region**: дефіцит Camp після локального виробництва покривається Food із запасу Castle без коефіцієнта транспортування.
- Якщо й запасу Castle недостатньо/немає — залишковий дефіцит компенсується Coins.
- Soldier coin upkeep залишається окремим і додається незалежно від food compensation.
- Transit — усе food consumption компенсується Coins.

Це також прибирає distance coefficient із військового постачання взагалі: він залишається тільки для потоків ресурсів/їжі між Region і Castle, де він і має природний сенс.

Для Castle Region ще треба буде коректно визначити food_balance : локальне виробництво Region + інші надходження в Castle – споживання Camp у Castle Region – інші витрати. Це вже нормально вкладається в Castle / CampInRegion без спеціальної логістики.

в Army та CampInRegion потрібно дати прямий зв'язок з Player.
Обчислення утримання буде організовано так:

Region має обчислювальні х-ки

food_consumption - споживання всіма військами

food_balance = виробництво - food_consumption

CampInRegion має обчислювальну характеристику

coins_food_compensation. Якщо camp в області замку -

coins_food_compensation=0. (компенсуцію буде рахувати сам

замок)

Є питання/зауваження?

Далі дивиться на `region.food_balance`. Якщо `food_balance >= 0`, `coins_food_compensation = 0`, інакше розраховує свою частку дефіциту

Castle рахує свій баланс - складає тільки позитивний баланс по всім областям з урахуванням відстаней, віднімає споживання будівлями і військами в своїй області. При негативному балансі - має `coins_food_compensation`

Зрештою, Player рахує суму `compensation` по всім своїм замкам, `CampInRegion` та арміям, що рухаються і враховує це у загальний банас монет.

Схема в цілому виглядає добре і, що важливо, дає односторонній потік залежностей без очевидного циклу.

Я бачу її так:

`Region.food_production` → `Region.food_balance` → `CampInRegion.coins_food_compensation`

і окремо:

позитивні `Region.food_balance` → `Castle.food_balance` → `Castle.coins_food_compensation`

а потім:

`CampInRegion + Castle + Movement` → `Player.coin_balance` .

Є кілька уточнень.

1. Прямий `player` у `Army` і `CampInRegion` підтримую. Для `Army` це прибирає необхідність визначати власника через `Knight/Commander/Camp`, а він потрібен у багатьох розрахунках. Для `CampInRegion` він взагалі є частиною сутності (`Player`, `Region`) .
2. `Region.food_consumption` має включати **всі Camp-війська всіх Player**, але не `Army` у `Transit`. Найприродніше рахувати його через `Region` -> `*camps[]` -> `CampInRegion.food_consumption` . Тобто `CampInRegion` потрібна власна `food_consumption` .

3. Для нейтральної Region формула частки дефіциту виходить простою. Якщо $\text{region.food_balance} < 0$, то для Camp:

$$\text{camp deficit} = (-\text{region.food_balance}) \times \text{camp.food_consumption} / \text{region.food_consumption}$$

і вже цей дефіцит множиться на coin_per_food .

Це автоматично дає пропорційний розподіл між Player.

4. Для **Castle Region** є важливе уточнення формулювання. $\text{Region.food_balance}$ уже дорівнює $\text{production} - \text{food_consumption}$. Тому Castle не повинен ще раз віднімати повне споживання військ своєї Region — інакше воно врахується двічі.

Я б визначив:

$\text{Castle.food_income} = \text{сума } \max(0, \text{region.food_balance}) \times \text{distance_coefficient}$ для його Regions.

Для самої Castle Region коефіцієнт = 1.

Але для Castle Region її негативний food_balance треба покривати запасом Castle. Тобто Castle має врахувати:

$$\text{castle_region_food_deficit} = \max(0, -\text{castle_region.food_balance})$$

плюс consumption будівель та інші Castle-level витрати.

Тоді ніякого подвійного віднімання немає.

5. Важливо: **негативний food_balance приєднаних звичайних Regions у Castle не передається взагалі**. Їхній дефіцит закриває $\text{CampInRegion.coins_food_compensation}$. У Castle йде тільки $\max(0, \text{region.food_balance})$.

6. Для CampInRegion у Castle Region $\text{coins_food_compensation} = 0$ правильно, навіть коли Castle порожній. Весь залишковий дефіцит агрегується вже в $\text{Castle.coins_food_compensation}$. Тобто не треба потім розподіляти його назад між Camp.

7. Для Movement я б теж дав $\text{computed coins_food_compensation}$, щоб на рівні Player була однакова схема:

$$\text{Player.food_compensation} = \sum \text{Castle.coins_food_compensation} + \sum \text{CampInRegion.coins_food_compensation} + \sum \text{Movement.coins_food_compensation}$$

А вже coin_balance враховує food_compensation разом зі звичайним coin upkeep Soldier та іншими доходами/витратами.

Єдине, що треба ще визначити перед внесенням у моделі: **споживання будівлями Food**. Ти зараз його згадав у `Castle.food_balance`, але я не пам'ятаю, чи ми вже остаточно визначили, які саме будівлі/процеси постійно споживають Food. Для структури достатньо `building_food_consumption`, а конкретні значення можуть залишитися в конфігурації.

4. Слушно - але простіше так: для Castle Region не віднімати споживання військ від власного production. Тобто, для Castle Region `food_balance = food_production`, а споживання віднімається уже самим замком.

7. Згоден

назви характеристик, які я дав в попередньому повідомленні можеш міняти на більш вдалі.

Доповнюй опис моделей (не тільки тим, що ми вирішили по споживанню їжі, а й інші доповнення, які ти знайшов при аналізі

Worked for 5m 15s

Оновив MODELS.md .

Крім нової Food-схеми, додав і попередньо знайдені прогалини:

- прямий `player` у `Army` і `CampInRegion`;
- `Player.*armies[]`, `Player.*camps[]`;
- `Region.food_production`, `camp_food_consumption`, `food_balance`, `food_surplus_to_castle`;
- `CampInRegion.food_consumption` і `food_coin_compensation`;
- `Castle.food_income`, `castle_region_army_food_consumption`, `food_balance`, `food_coin_compensation`;
- `Army.food_coin_compensation` для `Movement`;
- `eligible_camp_player_ids[]` окремо від загального `camp_player_ids[]`;
- `Region.*combat_situations[]` і pending combat/reinforcement logic;
- атаки `Player→Player` у `Neutral Camp`;
- точний retreat flow через `Movement.retreat-local`;
- lock combat parameters тільки в момент Battle Start;
- founder presence через `Knight.current_camp / is_regular_camp_presence`;
- raid-specific Neutral Defense combat;
- cleanup dead Knight з `Army`;
- safe Occupation restore через expected occupier;

- уточнення `Knight.location_state` : він не впливає на `Army/Camp logic`;
- автоматичний вихід `Knight` із `Barracks` при старті `Movement`.

Залишив явно позначеним один невизначений момент: окремий тимчасовий debuff `City income` після `Raid`. Зараз опис підтримує зміну `Wealth`; якщо потрібен незалежний тимчасовий penalty для `income`, треба окремо визначити його `recovery`-механіку.

Commit: 4fc29b26c3d0855fa733eb516b4ee90601cd10f1 .

City має в `wealth`:

`full_wealth` та `active_wealth` (назви можеш змінити).

В нормальному стані вони однакові і ростуть (коли місто нажить гравцю).

Після пограбування `active_wealth` падає до 0 після чого починає рости (швидше, ніж росте `full_wealth`, і не залежно від гравця).

Коли `active_wealth` відновлюється `full_wealth`, знов переходить в нормальний стан.

Ефективність пограбування та прибуток власника міста визначаються `active_wealth`. Можна `active_wealth` рахувати у відсотках відносно `full_wealth`. Тоді так - в нормальному стані `active_wealth = 100%`, `full_wealth` росте. Після пограбування `active_wealth` стає 0 і починає рости. `full_wealth` не росте, поки `active_wealth < 100%`. Окремо відслідковувати стан не потрібно, і явно розділяється ріст `full_wealth` та `active_wealth`.

Так, це чистіше. Окремий `status` для `City` тут справді не потрібен.

Я б назвав характеристики:

- `wealth` — абсолютний потенціал міста, те, що ти назвав `full_wealth` .
- `active_wealth_ratio` — від 0 до 1 (або 0–100%, але внутрішньо краще 0–1).

Тоді логіка така:

`wealth` росте, тільки якщо:

`active_wealth_ratio == 1` + `Region` належить `Player` + виконані інші умови нормального росту `Wealth`.

`active_wealth_ratio` :

- у нормальному стані = 1 ;

- при Raid встановлюється в 0 ;
- поки < 1 , росте зі своїм окремим rate;
- цей recovery не залежить від того, кому належить Region і чи належить вона взагалі комусь;
- при досягненні 1 просто перестає рости.

coin_income і raid_reward рахуються через ефективне Wealth:

$$\text{effective_wealth} = \text{wealth} \times \text{active_wealth_ratio}$$

Його навіть можна не вводити окремою характеристикою, якщо він ніде більше не потрібен.

Таким чином виходять дві незалежні динамічні характеристики:

wealth — довгостроковий розвиток міста;

active_wealth_ratio — короткострокове відновлення після пограбування.

І хороший побічний ефект: повторний Raid під час відновлення природно працює без додаткового стану — його ефективність визначається поточним active_wealth_ratio , після Raid ratio знову стає 0 .

Єдине, що треба визначити в правилах: чи будь-який Raid, навіть при active_wealth_ratio , наприклад, 0.02 , знову скидає його в 0 . Я б сказав так — тоді не потрібен мінімальний поріг або спеціальна заборона Raid.

звичайно, будь-яке пограбування скидає в 0. Нагадай, в нашій актуальній версії правил, грабувати місто, в тому числі нейтральне можна тільки з кемпу в його області?

Worked for 12s

Так. В актуальній версії правил **City Raid** — локальна дія: її можна почати тільки військами, які вже перебувають у Camp тієї самої Region , де знаходиться City.

Це стосується і нейтрального міста. Raid не можна запускати з іншої Region, під час Movement або Regrouping .

І так, фіксуємо додатково: **будь-який успішний Raid скидає active_wealth_ratio до 0** , незалежно від його поточного значення.

Деякі зміни по Neutral Defense та захисту міста.

Вводим окреме поняття - сила захисту міста [City Defense]. Вона визначається його wealth (не effective_wealth): City Defense =

f(wealth), конкретна функція задається в конфігурації. При пограбування для сторони захисника використовується саме ця сила, Neutral Defense не приймає участі в захисті міста. Місто, що належить гравцю, все ще має City Defense.

При атаці на Neutral Defense області з метою її подальшого приєднання до сили Neutral Defense додається City Defense. Якщо гравець переміг - Neutral Defense вважається знищеним, City Defense не змінюється, але й не заважає таймеру приєднання, окремо ще раз перемагати City Defense для приєднання не потрібно.

Зміна по відновленню Neutral Defense - воно відновлюється поступово, до попереднього значення, відновлення починається після того, війська всіх гравців залишили область, при цьому воно одразу стає > 0 . Тобто, якщо після розгрому Neutral Defense війська гравця вийшли з області і одразу повертаються - буде новий бій, при цьому Neutral Defense буде значно менше, бо ще не відновилося, а City Defense буде таким же як і раніше.

Можна для відновлення Neutral Defense використовувати такий же принцип, як і для City Wealth, але з поправкою - починає відновлюватися тільки коли вийшли військ, при цьому одразу стає > 0 .

При атаці на область гравця, City Defense не долучається до оборони, тільки захищається від пограбування (а грабувати можна тільки після того, як війська гравця в цій області переможені).

Ще раз підсумуємо:

- вхід в в кемп в нейтральну область: без бою
- пограбування міста в нейтральній області з кемпу: в обороні тільки City Defense
- атака на Neutral Defense: в обороні Neutral Defense + City Defense (це правило не дуже логічно, але воно реалізує вимогу "захопити область з містом має бути складніше, ніж просту область". Якщо вважаєш зайвим ускладненням - можна просто робити Neutral Defense областей з містом більше, ніж у звичайних областей).

- атака на область гравця з містом: в обороні тільки армії гравця
- пограбування міста, що належить гравцю: спочатку треба перемогти армії гравця, стати в кемп - після того атакувати City Defense. Для приєднання області, що вже належить іншому гравцю, перемагати City Defense не потрібно.

Так, схема зрозуміла. Я б залишив саме окремий City Defense , а не просто підвищував Neutral Defense для областей із містом.

Причина в тому, що це вже дві різні за змістом величини:

- Neutral Defense — оборона самої нейтральної Region, яка може бути знищена й поступово відновлюється.
- City Defense — постійна обороноздатність City, що визначається wealth , не знищується в бою і безпосередньо захищає City від Raid.

Тоді те, що під час атаки Neutral Defense використовується Neutral Defense + City Defense , можна трактувати просто як спеціальне правило захоплення Region із City. Реалізаційно воно теж нескладне: CombatSituation для neutral_defense бере дві складові defensive strength, але результат перемоги змінює тільки neutral_defense .

Щодо поступового відновлення Neutral Defense я б не робив відсоткову характеристику, як для City, бо тут у нас уже є сама числова neutral_defense . Достатньо:

neutral_defense — dynamic, поточна сила;

neutral_defense_max — computed/constant-derived нормальна сила Neutral Defense цієї Region;

neutral_defense_recovery_rate — computed rate:

- 0 , поки в Region присутні війська хоча б одного Player;
- > 0 , коли військ немає і neutral_defense < neutral_defense_max ;
- 0 при досягненні максимуму.

Після програного бою neutral_defense = 0 . Коли останні війська залишають Region, rate стає позитивним. Через нашу модель dynamic characteristics при наступному зверненні через будь-який $\Delta t > 0$ значення вже буде > 0 . Тобто окремий штучний стрибок з 0 не потрібен.

Виходить така матриця:

- Neutral Region → вхід у Camp: бою немає.
- Neutral Region + Raid City → Defender = City Defense .

- Neutral Region + attack Neutral Defense → Defender = Neutral Defense + City Defense , якщо City є.
- Owned Region → вторгнення: Defender = війська owner; City Defense не бере участі.
- Owned Region + Raid City → спочатку треба отримати можливість перебувати в Camp після перемоги над військами owner; потім окрема атака, Defender = City Defense .
- Annexation Neutral Region → достатньо один раз перемогти Neutral Defense + City Defense ; після цього City Defense окремо не блокує Annexation.
- Annexation чужої Owned Region → City Defense взагалі не є умовою Annexation.

Є лише один нюанс, який я б зафіксував зараз: **City Defense при Raid теж не зменшується після бою**, правильно? Тобто навіть після успішного Raid $city_defense = f(wealth)$ залишається тією самою, бо $wealth$ не змінюється — падає тільки $active_wealth_ratio$. Відповідно повторний Raid через короткий час знову вимагає перемогти повну City Defense, хоча потенційна здобич буде майже нульовою.

Якщо так — механіка повністю однозначна і добре вкладається в поточну архітектуру.

саме так, City Defense не залежить від попередніх атак. І залишаються пороги бою. При пограбуванні міста - City Defense використовує фіксований поріг втрат (30-50%, задається в конфігурації), а при атаці на Neutral Defense - Neutral Defense та City Defense використовують поріг 100%

Тоді фіксуємо.

При City Raid :

- defender — тільки City Defense ;
- City Defense = $f(wealth)$, не залежить від $active_wealth_ratio$;
- для City Defense використовується фіксований $loss_threshold$, заданий у конфігурації (орієнтовно 30–50%);
- після бою City Defense не зменшується;
- при успішному Raid $active_wealth_ratio \rightarrow 0$, $wealth$ не змінюється.

При атаці Neutral Defense :

- defender strength = Neutral Defense + City Defense , якщо в Region є City;
- для **обох складових** $threshold = 100\%$;
- отже, для перемоги потрібно повністю подолати всю сумарну оборону;
- після перемоги Neutral Defense $\rightarrow 0$;
- City Defense не змінюється;

- повторно перемагати City Defense для Annexation не потрібно.

Це також означає, що один і той самий City Defense бере участь у двох типах CombatSituation з різними thresholds. Тому threshold не повинен бути характеристикою самого City. Для Raid він береться з конфігурації raid combat, а для атаки Neutral Defense примусово дорівнює 100%.

Архітектурних проблем тут не бачу. Це добре вкладається в наявний CombatSituation через combat_type і зафіксовані combat parameters.

внеси зміни в правла і в опис моделі, в правила також внеси зміну по утриманню військ

Worked for 2m 25s

Analysis paused

Так. У 01_GAME_RULES_V1 треба внести три узгоджені блоки змін:

1. Нова схема Food/upkeep: локальна Food у Camp, пропорційний дефіцит між гравцями, компенсація Coins; без постачання Food із Castle у віддалені Region; виняток — Castle Region; Movement повністю компенсує Food через Coins; звичайний Soldier Coin upkeep окремий.
2. City : wealth + active_wealth_ratio , City Defense = f(wealth) , Raid скидає ratio в 0, City Defense при Raid має конфігураційний threshold, не зменшується після атак.
3. Neutral Defense : при атаці для захоплення Defender = Neutral Defense + City Defense, обидва з threshold 100%; City Defense після бою не змінюється; Neutral Defense після знищення поступово відновлюється до попередньої/нормальної сили тільки після виходу всіх військ.

У MODELS.md відповідно треба переробити Region , City , CombatSituation і пов'язані methods/triggers, зокрема прибрати старий neutral_defense_recovery_complete .

Я ще не закомітила ці зміни: поточний GitHub-конектор дозволяє замінювати файл тільки цілком, а обидва документи великі; не хочу ризикувати пошкодити незмінені частини при повній ручній перезаписі. <>

які пропозиції?

Я б запропонувала не переписувати файли вручну через update_file , а зробити зміни як нормальний patch через GitHub/Codex: змінити тільки потрібні фрагменти у двох файлах, перевірити diff і зробити один commit.

Це найкращий варіант, бо:

- не чіпає решту великих документів;
- можна одразу побачити точний diff;

- легше перевірити, що правила й `MODELS.md` не суперечать одне одному;
- одним комітом внесемо і нову логіку `City/Neutral Defense`, і зміну утримання військ.

Можу зараз піти саме цим шляхом.

Для `Game rules` створи окремий файл `"rules_update"`, опиши зміни, потім зберем в один.

Thinking

Створив `docs/rules_update.md` у `main`.

В ньому окремо описані погоджені зміни:

- `Food consumption` та компенсація `Coins`;
- виняток для `Castle Region`;
- утримання `Army` у `Movement`;
- `wealth / active_wealth_ratio` ;
- `City Defense` ;
- `City Raid`;
- участь `City Defense` в атаці `Neutral Defense`;
- поступове відновлення `Neutral Defense`.

Commit: `3af6d7db4a4553a1956710f65baeb97d540f6c81`.

`01_GAME_RULES_V1` не змінював. Наступним окремим запитом можемо оновити `MODELS.md`, дотримуючись правила «один запит — один commit».

тепер онови `MODELS.md`

Worked for 4m 14s

Оновив `docs/MODELS.md`.

Основні зміни:

- `Region.neutral_defense` тепер `dynamic` і відновлюється поступово;
- додані `neutral_defense_full_strength` та `neutral_defense_recovery_rate` ;
- `City` тепер має `wealth + active_wealth_ratio`, `effective_wealth`, `city_defense` ;
- `Raid` захищається тільки `City Defense` ;
- атака `Neutral Defense` в `Region` з `City` використовує `Neutral Defense + City Defense` ;
- для `Neutral Defense threshold` = `100%`, для `City Raid` — окремий конфігураційний `threshold`;
- `City Defense` ніколи не зменшується від боїв;

- `CampInRegion.can_progress` і `CastleFounding` явно вимагають `neutral_defense == 0` у Neutral Region;
- прибрано стару модель «чекаємо recovery time і відновлюємо Neutral Defense одразу повністю».

Commit: 6ae2f326eaf21361176b7f5ce2dd00b57f57928e .

тепер ще раз проаналізуй опис моделей. Тепер він покриває всю логіку?

Worked for 2m 1s

Ще не повністю. Після останніх змін основна структура вже покривається, але я бачу кілька конкретних прогалин.

Найважливіші:

- `City.user_raid_city(camp)` не визначає, яка саме `Army` є `attacker`. Те саме з `CampInRegion.user_attack_player(target_camp) : CombatSituation.attacker` — одна `Army`, але метод вибирає лише `Camp`. Треба передавати `attacker_army` і валідовувати, що вона належить цьому `Camp` та не `Regrouping`.
- Для `Annexation` `home_castle_ids[]` зараз включає і `Regrouping Army`. Через це `Army` з потрібного `Castle` у `Regrouping` може формально задовольнити вимогу. Потрібен окремий `eligible_home_castle_ids[]`, тільки для `state == Camp`.
- `Occupation` зараз відновлюється в `CampInRegion.on_army_left()`, тобто коли остання `Army` залишила `Camp`. Але `Army` на початку `Movement` ще фізично знаходиться в `Region`. За правилами контроль має повертатися, коли окупант **повністю залишив Region**, тобто на перетині кордону, а не при виході з `Camp`.
- `neutral_defense_recovery_rate` зараз не перевіряє, що `Region` саме `Neutral`. Формально `Neutral Defense` може почати відновлюватися і в `Owned Region` без військ. Треба додати `player == null` як умову `recovery`.

Економіка має ще два недопрацьовані місця:

- `food_surplus_to_castle` треба явно обнулити при `Occupation` і при `is_connection_valid == false`. Для `Wood/Stone/Iron` це прямо записано, для `Food` зараз ні.
- `Player.gold_balance / silver_balance` досі агрегують фактичний `production Region` напряму. Це означає, що формальний `owner` теоретично продовжує отримувати `Gold/Silver` з `Occupied Region`. Потрібні характеристики `Region` на кшталт `gold_income_for_owner / silver_income_for_owner`, які враховують `occupation/connectivity`.

Ще одна архітектурно важлива дрібниця — `Retreat.get_legal_retreat_regions()` має розрізняти війська, які реально блокують `Retreat`, від `Army`, що просто проходить `Region` у `Transit`. Зараз у `Region` є `*armies[]` і загальний `has_any_troops`, але немає готової характеристики «blocking/local military presence». Я б підняв це в `Region`, наприклад як `retreat_blocking_player_ids[]`, щоб `CombatSituation` не мусив проходити ланцюг `CombatSituation → Region → Army → Movement`.

І треба явно визначити `attacker threshold` для абстрактних боїв. Для `Neutral Defense` старі правила вже фактично задавали `Target Combat Threshold`; для `City Raid` зараз у моделі визначений `defender threshold City Defense`, але не зафіксовано, який із трьох `threshold Army` використовується як `attacker threshold`.

Отже, я б сказав: **приблизно 90–95% логіки вже покрито; фундаментальної проблеми архітектури немає**. Залишилися переважно точні `relationships/derived characteristics` і кілька `lifecycle-переходів`. Найсерйозніше зараз — `Occupation exit`, `attacker Army` у `Raid/neutral PvP` і `Gold/Silver flow`.

`user_raid_city` та `user_attack_player` поки просто перенесем в `army`.

Що таке `Annexation` `home_castle_ids[]` ?

Так, `user_raid_city` і `user_attack_player` логічніше перенести в `Army`: тоді `Army` однозначно є `attacker`, і не треба окремо передавати/визначати її через `Camp`.

Щодо `home_castle_ids[]`: це характеристика `CampInRegion`, яку ми ввели для `Annexation`. Вона містить ID рідних замків усіх `Knight`, чий `Army` входять у цей `Camp`.

Вона потрібна через наше правило: **приєднати `Region` до конкретного `Castle` можна тільки якщо в `Camp` є хоча б один `Knight`, для якого цей `Castle` є рідним**.

Ланцюжок такий:

`CampInRegion → armies[] → Army.home_castle_ids[] → Knight.castle`

А `Castle.can_annex_region(region, camp)` перевіряє, чи є ID цього `Castle` у `camp.home_castle_ids[]`.

Моє зауваження було в тому, що зараз `home_castle_ids[]` включає `Knight` з усіх `Army` у `Camp`, включно з `Regrouping`. Але ми вже визначили, що `Regrouping` не дає `presence` для `Annexation`. Тому правильніше або рахувати `home_castle_ids[]` тільки по `Army` зі `state == Camp`, або мати окремий `eligible_home_castle_ids[]`.

Я б вибрала перший варіант: **не вводити ще одну характеристику**, а визначити `CampInRegion.home_castle_ids[]` тільки за Army, які дають `has_eligible_presence`.

`Castle.can_annex_region` - це ж не обчислювальна характеристика? Метод може використовувати будь-які зв'язки без обмежень, тому для нього взагалі не обов'язково городити обчислювальні характеристики

Так. Тут моє попереднє зауваження було зайвим.

`Castle.can_annex_region()` — звичайний domain method, а наше обмеження на ланцюги зв'язків стосується **обчислювальних характеристик**, не методів.

Тому метод може без проблем пройти:

`Castle → Region → CampInRegion → Army → Knight`

і перевірити без додаткових computed-характеристик:

- що є потрібний `CampInRegion` ;
- що в ньому є Army у допустимому для Annexation стані (`Camp` , не `Regrouping`);
- що серед Knight цих Army є хоча б один із `knight.castle == this Castle` .

Отже `CampInRegion.home_castle_ids[]` для цієї задачі взагалі не потрібна. Якщо вона більше ніде не використовується як характеристика — її варто прибрати з `MODELS.md` .

Це важливий принцип для подальшого рев'ю: **не створюємо проміжні computed-характеристики лише для спрощення domain method**. Вони потрібні насамперед для dynamic characteristics / triggers або коли значення саме по собі є корисною характеристикою моделі.

Occupation припиняється при виході з кемпу.

Так. Тоді моє зауваження щодо Occupation теж знімається.

Фіксуємо: **Occupation існує, поки війська occupier перебувають у Camp**. Коли остання Army occupier виходить із Camp для початку Movement, Occupation одразу припиняється і контроль повертається формальному owner — чекати фактичного перетину кордону Region не потрібно.

Отже поточна логіка через `CampInRegion.on_army_left()` правильна.

ще щось є, що не можеш вирішити без моєї участі?

Із тих прогалин, які я знайшла під час останнього ревію, після твоїх уточнень залишається фактично одне питання, де потрібне твоє рішення:

Який attacker threshold використовується для Army при City Raid?

Для defender ми вже визначили:

- City Defense має фіксований threshold із конфігурації, приблизно 30–50%.

Але Army attacker теж повинна мати threshold, за яким визначається її поразка/вихід із бою.

Варіанти: її звичайний Target Combat Threshold, який Player задає для Army, або спеціальний фіксований threshold для Raid.

Я б використовувала звичайний Target Combat Threshold Army. Тоді Raid нічим особливим не відрізняється для атакуючих військ.

Решту можу виправити без додаткових рішень:

- перенести user_raid_city і user_attack_player в Army;
- прибрати зайві home_castle_ids[], якщо вони ніде більше не потрібні;
- Neutral Defense recovery дозволяти тільки для Neutral Region;
- уточнити Food flow при Occupation/disconnection;
- виправити Gold/Silver flow, щоб Occupied Region не давала дохід формальному owner;
- за потреби додати Region-характеристику для military presence, потрібну Retreat, не створюючи зайвих computed-характеристик для domain methods.

Тобто після рішення про Raid attacker threshold я можу зробити наступну ітерацію MODELS.md самостійно.

так, використовуй звичайний Target Combat Threshold Army і онови опис моделей

Worked for 4m 35s

Оновив MODELS.md.

Змінив:

- user_raid_city і user_attack_player перенесені в Army;
- для City Raid attacker використовує звичайний Target Combat Threshold;
- прибрані home_castle_ids[] з Army / CampInRegion як зайві для Annexation;
- Castle.can_annex_region() тепер напряму проходить CampInRegion → Army → Knight;
- Neutral Defense recovery працює тільки для Neutral Region;

- уточнений Food flow при Occupation/disconnection;
- Gold/Silver більше не надходять owner з Occupied/disconnected Region;
- Retreat validation напряду аналізує Army/Movement у сусідніх Region без штучної computed-характеристики;
- явно зафіксував, що Occupation припиняється при виході останньої Army occupier із Camp.

Commit: 72296fc93f3333d37b6d2d59ae2d5f63545fff97 .

проаналізуй ще раз

Worked for 3m 58s

Ще не повністю. Після повторного проходу бачу кілька реальних прогалин.

1. Найважливіша — у Movement не зберігається зафіксований локальний режим Transit / Aggressive Transit. Зараз є лише phase = transit / final-local / retreat-local. Але за правилами Aggressive Transit визначається при вході в Region і після цього не повинен змінюватися, навіть якщо стан кінцевої Region змінився. Тут потрібна пряма характеристика на кшталт local_mode.
2. Не дописаний випадок, коли під час руху майбутня Region стає недоступною, особливо якщо в ній завершується Founding Castle. validate_route() перевіряє Route при його заданні, але перед фактичним входом у next_region повторної перевірки явно немає. А правило grandfathering дозволяє завершити Transit через Castle Region тільки якщо Army вже була всередині неї.
3. Ми маємо правило: коли Army вже рухається до Camp у кінцевій Region, гравець може заздалегідь задати наступний Route. У моделі немає чітко відокремленого queued/post-arrival route і немає переходу «досяг Camp → завершив interaction → автоматично почав запланований наступний Movement». Поточний user_change_planned_route() цього явно не покриває.
4. Для neutral_camp_player_combat треба явно записати, що Defender теж використовує Army.attack_strength. Зараз defender_strength для player combat описаний узагальнено, тому правило «в Neutral Camp обидві сторони використовують Attack» не зафіксоване достатньо чітко.

Є ще два моменти, де вже потрібне твоє рішення:

- При програвші City Raid куди Retreat-ить Army? Для атаки Neutral Defense ми вже мали правило «будь-яка з 6 сусідніх Region». Для City Raid це ще не визначали. Я б застосував те саме правило — combat починається з Camp, тому напряду входу немає.
- Якщо в CampInRegion накопичений Annexation progress, усі Army у звичайному Camp зникають, але лишається тільки Regrouping Army — progress зберігається чи скидається? Зараз

модель його зберігає, бо `CampInRegion` не закривається, доки в ньому є хоч одна `Army`.

Решта попередньо знайдених проблем зараз закрыта. Фундаментальної проблеми архітектури не бачу.

Monday 9:59 PM

`Regrouping` `Army` для `Annexation` рахується як звичайна армія в кемпі. Єдине обмеження для `Regrouping` `army` - не може рухатись і атакувати

Тоді моє друге питання знімається.

Фіксуємо загальне правило: `Regrouping` — це повноцінна `Army` у `Camp` для всієї логіки `Camp` `presence`. Вона:

- рахується для `Annexation` і не зупиняє/не скидає `control_progress` ;
- рахується для `Castle Founding` `presence`;
- бере участь у `defense`;
- споживає `Food` як `Army` у `Camp`;
- впливає на `Occupation` та існування `CampInRegion` .

Єдині спеціальні обмеження `Regrouping` : **`Army` не може почати `Movement` і не може ініціювати `Attack`.**

Отже, поточні `has_eligible_presence` , `eligible_camp_player_ids[]` , `founder_progress_eligible` та пов'язані умови в `MODELS.md` треба спростити: вони зараз помилково виключають `Regrouping`.

Залишається невирішеним моє питання про `Retreat` після програного `City Raid` .

після програного `City Raid` залишається там де була

Тоді для `City Raid` `Retreat` взагалі немає.

При програному `Raid`:

- `Army` залишається в тому самому `CampInRegion` ;
- не переходить у `Movement` чи `Regrouping` лише через факт поразки;
- отримані втрати застосовуються звичайно;
- `City` не пограбоване, `active_wealth_ratio` не скидається;
- `City Defense` не змінюється.

Тобто City Raid — виняток із загальної логіки програшу атакуючої Army: CombatSituation для city_raid не викликає start_retreat_to() .

Це також спрощує модель: retreat destination для city_raid взагалі не треба обчислювати чи lock-ити.

Regrouping застосовується, але в тій же області

Так, тоді уточнення:

При програному City Raid Army:

- залишається в тому самому CampInRegion ;
- переходить у Regrouping ;
- після завершення Regrouping повертається у звичайний Camp ;
- під час Regrouping не може рухатися або атакувати, але для решти Camp-логіки залишається звичайною присутньою Army: Annexation, Occupation, Food consumption, defense тощо продовжують діяти.

Тобто це відрізняється від звичайного програшу в combat: **немає Retreat, лише локальний Regrouping після поразки.**

так. Нагадай наші правила по долученню до бою армій захисника, що рухаються по атакованій області на момент входу атакуючого, або входять вже після входу атакуючого

Так, ми розділяли це за моментом відносно Battle Start :

- Якщо Army захисника вже рухається (Transit) по Region у момент входу attacker, вона **не долучається автоматично**. До Battle Start захисник може зупинити її Transit і долучити до defense. Для цього у нас є user_stop_transit_for_defense() .
- Якщо Army захисника входить у атаковану Region **після attacker, але до Battle Start**, вона автоматично долучається до pending CombatSituation як reinforcement захисника. Її рух припиняється.
- У момент Battle Start склад сторін фіксується (locked_combat_parameters).
- Army, яка входить **після Battle Start**, до вже розпочатого бою не долучається. Для неї ситуація має оброблятися вже за актуальним станом Region після/окремо від поточного бою.

Також для Army, яку захисник вручну зупинив із Transit для defense, це спеціальний виняток із нормального руху: Movement.stop_for_defense() → CombatSituation.add_defender() .

Саме так це зараз описано в `MODELS.md` : є окремий `user_stop_transit_for_defense()` , автоматичне reinforcement при arrival до Battle Start і lock складу стопін на Battle Start.

Останній пункт — що саме робити з Army, яка входить **після Battle Start, але до завершення бою**, — наскільки бачу, ми детально ще не формалізували.

Monday 11:25 PM

Внесемо уточнення, можливо та зміни.

У всіх правилах нижче Regrouping рахується за стан Camp. Коли кажемо мета атакуючого - транзит, мається на увазі транзит або агресивний транзит. Там, де тільки неагресивний транзит - це прямо вказується.

Армія гравця A входить в область гравця B.

Умови, за яких бій не відбувається:

1. Атакуючий входить з метою транзит, на момент входу атакуючого в області немає армій гравця B в стані Camp , або таких, що уже ввійшли в область для Camp: бойової ситуації НЕ ВІНИКАЄ*. Атакуючий проходить транзитом, якщо є армії захисника в транзиті - теж проходять, ніхто не зупиняється, гравцю B не показується діалог підготовки до бою. *Фактично, модель CombatSituation може бути створена для оборобки входу, але одразу видаляється.

2. Атакуючий входить з метою неагресивний транзит, а гравця B є армії, які знаходяться в Camp або готуються стати в Camp: перше, що пропонується гравцю B - вирішити, чи пропускати транзитну армію без бою. У вікні 2 кнопки "Пропустити" і "Прийняти бій" (`allow_transit` використовується на випадок, коли гравець не онлайн, або не прийняв рішення вручну). Після натискання одної з кнопок, вибір фіксується, змінити його вже не можна. Якщо вибір "пропустити" - на цьому бойова ситуація стає вирішена, атакуючий проходить, на армії захисника ніяк не впливає. Вибір "Прийняти бій" - бойова ситуація залишається в

силі.

Якщо бойова ситуація не вирішилась умовами 1 або 2:

Правила для армій гравця В, які знаходились або увійшли в область до входу атакуючого, не залежить від мети атакуючого:

- армія в Camp не момент входу - може залишитись для бою або відступити (з початком бою)
- армія вийшла з Camp до входу атакуючого - не може залишитись, продовжує рух, виходить з області без перешкод
- армія увійшла для Camp - може залишитись для бою або відступити (з початком бою)
- армія увійшла для транзиту - може залишитись для бою (таке рішення має бути прийнято до її виходу з області) або продовжити транзит

Правила для армій гравця В, які увійшли в область після входу атакуючого, залежать від мети атакуючого та результату бою:

- мета атакуючого транзит, будь-який результат бою: всі армії гравця В, що входять після входу атакуючого, продовжують рух по запланованому маршруту, в т.ч. можуть стати в кемп (це відбудеться вже після бою), не можуть долучитися до бою
- мета атакуючого кемп, атакуючий програв: продовжують рух по запланованому маршруту, в т.ч. можуть стати в кемп
- мета атакуючого кемп, атакуючий виграв і окупував область: діє правило входу на свою територію, окуповану іншим гравцем, тобто, армія В, яка входить в цю області після атакуючого має сама атакувати окупанта, незалежно від своєї початкової мети. Але в цьому випадку бій відбувається через Dt після першого бою. Вводим загальне правило: в одній області не можуть відбутися 2 бою з інтервалом менше Dt од. часу між ними, бойові ситуації "стають в чергу". Виключення - бій на нейтральній території між різними парами гравців, атаки на нейтральнів та пограбування міста різними арміями.

Так, схема зрозуміла. Вона суттєво точніше визначає, хто саме потрапляє в CombatSituation. Я б її прийняла майже без змін.

Є кілька моментів, які варто явно зафіксувати, щоб потім не виникла неоднозначність.

По-перше, момент «увійшла в **Region** для **Camp**». Я трактую таку Army як таку, що вже committed to Camp arrival, навіть якщо вона ще проходить `final-local` і фізично Camp не досягла. Тому вона прирівнюється до Camp Army для визначення наявності захисту та може бути учасником майбутнього бою. Це відповідає твоєму формулюванню.

По-друге, для умови №1 виходить важливий наслідок: якщо attacker іде Transit/Aggressive Transit, а у B є тільки Transit Army, `CombatSituation` фактично не починається. Отже B **не може спеціально зупинити Transit Army для перехоплення attacker**. Це відрізняється від попередньої версії, де ми дозволяли `stop_transit_for_defense()`. Тепер така можливість існує тільки якщо `CombatSituation` вже виникла з іншої причини — через Camp/Camp-bound Army B.

По-третє, для умови №2 `allow_transit` я б трактувала саме як **fallback decision**, а не автоматичне рішення в момент входу:

`manual decision` → якщо його немає до `decision deadline` → `allow_transit`.

Треба лише пізніше визначити, скільки Game Time дається B на це рішення. Це може бути той самий pre-battle preparation interval або окремий конфігураційний параметр.

Нова класифікація Army B на момент входу A тоді виходить дуже чистою:

- `Camp / Regrouping` → potential defender;
- `final-local` → `Camp` → potential defender;
- уже вийшла з `Camp` → не defender;
- `Transit` → optional defender, але тільки якщо `CombatSituation` вже існує;
- увійшла після attacker → **ніколи не reinforcement цієї `CombatSituation`**.

Останній пункт змінює наше попереднє правило про автоматичне приєднання reinforcement до pending combat. Його треба прибрати з `Region.resolve_arrival()` / `CombatSituation.add_defender()`.

Щодо `Dt` теж підтримую. Я б тільки сформулювала його не як затримку між `CombatSituation`, а як мінімальний інтервал між **Battle Start** у межах **Region**. Інакше `CombatSituation` можна створювати одразу, ставити в queue і показувати гравцю, але її **Battle Start** не може відбутися раніше:

`previous_battle_start + Dt`

або, можливо, ти маєш на увазі **Dt після завершення попереднього бою**. З твого «бій відбувається через Dt після першого бою» я скоріше розумію саме:

$$\text{next_battle_start} \geq \text{previous_battle_end} + Dt .$$

Це треба уточнити.

Винятки я розумію так: обмеження Dt не застосовується між незалежними combats у Neutral Region, якщо вони не конфліктують між собою — різні пари Player, різні Army атакують Neutral Defense/City тощо. Тобто глобальної «одна Region = один combat» блокування для Neutral Region немає.

І ще один важливий наслідок: якщо А виграв, став у Camp і створив Occupation, а B1/B2/B3 уже рухаються до цієї Region, **кожна з них при вході створює нову атаку на occupier**, навіть якщо спочатку мала Transit . Якщо таких Army кілька, відповідні CombatSituation можуть накопичитися в черзі через Dt .

Архітектурно все це вирішується поточною системою. Але перед внесенням у MODELS.md я б уточнила тільки одну річ: **Dt відраховується від Battle Start чи від завершення попереднього бою?** Я б відраховувала від завершення — це природніше для правила «перерва між боями».

скільки Game Time дається B на це рішення - той самий Dt. Dt - одна константа для:

- час проодження одної області транзитом*
- час від входу в область до переходу в Camp
- час від виходу з Camp до входу в іншу область
- час від входу в область до початку бою
- час підготовки до атаки з кемпу в нейтральній області (не застосовується для атаки на нейтралів та пограбування міста)
- час від поразки в бою до входу в Camp-regrouping (не застосовується для поразки від нейтралів або міста)
- час самого regrouping також зробим Dt, щоб не вводити додаткових параметрів
- мінімальний час між боями в області

*якщо вирішимо робити змінну швидкість армій - Dt буде базова швидкість, яка може бути покращена.

І так, якщо в області декілька CombatSituation - діалогове вікно наступного бою показується гравцю після закінчення попереднього бою, таким чином час на прийняття рішення завжди фіксований = Dt. Початком бойової ситуації вважається час закінчення попереднього бою. Тут всеодно ще треба буде формалізувати правила, зробим пізніше.

Так. Тоді Dt фактично стає **базовим часовим квантом просторових і бойових дій**, а не набором окремих балансних параметрів. Це значно спрощує конфігурацію.

Я б зафіксувала одну важливу термінологічну різницю: Dt — базова тривалість, але не обов'язково фактична тривалість кожного процесу. Наприклад, якщо пізніше введемо швидкість Army:

$$\text{transit_time} = Dt / \text{movement_speed_coefficient}$$

При цьому decision window перед боєм все одно може залишатися рівно Dt. Тобто не варто прив'язувати всі ці процеси до однієї computed-функції часу — вони просто використовують одну конфігураційну константу.

Щодо черги CombatSituation тепер зрозуміло. Якщо попередній бій закінчився в T, наступна queued CombatSituation фактично активується в T, саме тоді показується її діалог, а:

$$\text{Battle Start} = T + Dt.$$

Таким чином гравець завжди має повний Dt на рішення. До моменту активації queued CombatSituation її умови можуть уже змінитися, тому при активації треба буде заново перевіряти її validity. Детальну queue-логіку справді краще формалізувати окремо пізніше.

Є лише одна річ у твоєму переліку, яку важливо не загубити: два винятки після поразки.

При звичайному бою:

$$\text{defeat} \rightarrow Dt \text{ руху до Camp} \rightarrow \text{Camp/Regrouping} \rightarrow Dt \text{ regrouping}.$$

При поразці від Neutral Defense або City Defense перший Dt **не застосовується**. Тобто Army одразу залишається/опиняється у своєму локальному Camp-контексті та починає Regrouping тривалістю Dt. Для City Raid це точно відповідає щойно погодженому правилу.

Тепер для внесення попередніх змін у MODELS.md мені додаткових рішень не потрібно.

Формалізацію кількох queued CombatSituation можна залишити явно позначеною як ще не

деталізовану.

ще не все.

Не описали, що відбувається, якщо B не прийняв ніяких рішень до початку бою.

- мета атакуючого неагресивний транзит, `allow_transit=true`: бій не відбувається, атакуючий продовжує транзит, всі армії B також залишається в своїх поточних станах.

- бій відбувається:

- армії, які були в транзиті - продовжують транзит

- армії, які ввійшли в кемп до бою - якщо поріг втрат в обороні

0% - відступають з регрупінг, але без втрат, інакше - приймають участь в бою.

Так. Тоді для відсутності ручного рішення B до завершення `Dt` маємо чіткий default resolution.

Якщо A йде в неагресивний Transit і `B.allow_transit = true`, CombatSituation завершується без Battle:

- A продовжує Transit;
- усі Army B залишаються у своїх поточних станах;
- ніяких Retreat/Regrouping для B немає.

В усіх випадках, коли Battle все-таки має початися, на Battle Start автоматично визначається склад Defender:

- Army B, яка перебуває в Transit → **не бере участі**, продовжує Transit;
- Army B, яка на цей момент уже досягла Camp (включно з Regrouping) і має `defense_loss_threshold > 0` → **бере участь у Battle**;
- Army B у Camp з `defense_loss_threshold = 0` → **автоматично Retreat без бою і без втрат**, після чого проходить звичайний post-retreat Regrouping;
- Army B, яка була `final-local` → Camp на момент входу A і до Battle Start уже досягла Camp, відповідно потрапляє під ті самі правила.

Тут є одна деталь, яку ще треба визначити: **що робити з Army B, яка на момент входу A вже ввійшла в Region для Camp, але через `Dt` руху до Camp ще не встигла його досягти на момент Battle Start.**

Якщо вона ввійшла одночасно або раніше за A, то за однакового `Dt` вона мала б досягти Camp не пізніше Battle Start. Але можуть бути boundary-випадки або майбутня змінна швидкість. Тому краще

зараз явно визначити правило для такої Army, а не покладатися на рівність часів.

НЕ можуть бути boundary-випадки або майбутня змінна швидкість. Час від входу до кемп та до початку бою завжди однаковий = Dt, змінюватись може тільки час транзиту без бойової ситуації

Так, тоді додаткового випадку немає. Моє зауваження знімається.

Фіксуємо жорстко:

entry into Region → Dt → Camp

i

CombatSituation start → Dt → Battle Start .

Тому будь-яка Army B, яка **ввійшла в Region для Camp до або одночасно з входом A**, гарантовано досягне Camp не пізніше Battle Start. Її подальша поведінка на Battle Start визначається вже як для Army у Camp.

Змінна швидкість у майбутньому може впливати **тільки на Transit без CombatSituation**. Вона не змінює:

- Region entry → Camp = Dt ;
- Camp exit → next Region entry = Dt ;
- CombatSituation start → Battle Start = Dt ;
- post-defeat movement = Dt ;
- Regrouping = Dt ;
- інші зафіксовані нами процеси на Dt .

Отже, спеціальної логіки для «ще не встигла дійти до Camp до Battle Start» нам не потрібно.

Бій між гравцями в нейтральній області:

A та B знаходяться в нейтральній області в Camp. A ініціює атаку на B. Виникає бойова ситуація, час до початку бою = Dt. B отримує діалогове вікно, схожу на вікно захисника при атаці.

- може прийняти рішення відступити повністю, або окремо для частини армії (відступ без втрат але з регрупінг, по результату бою а не миттєвий)
- може встановити поріг втрат в обороні (один для всіх армій що

залишаються для бою)

Якщо до виникнення бойової ситуації в області увійшла для кемпа інша армія гравця В - вона приймає участь в бою або відступає, на тих самих умовах, що й армії які вже були в кемпі. Армія гравця В, яка увійшла для транзиту, та армія, яка вже вийшла з кемпу в цій області - не приймають участь в бою. Армія гравця В, яка увійшла після виникнення бойової ситуації не може прийняти участь в бою.

Після виникнення бойової ситуації В не може дати своїй армії, яка була в кемпі, команду на переміщення. Вона може або прийняти участь в бою, або відступити без втрат з регрупув по завершенні бою.

Ті самі правила діють для армій, що окуповують чужу область, при атаці на них з боку власника області або третього гравця.

Так. Це добре уніфікує player-vs-player combat.

Фіксує для Neutral Camp attack :

- У момент ініціації атаки створюється `CombatSituation` ; `Battle Start` = `+Dt` .
- Склад потенційних defenders фіксується за присутністю **на момент виникнення `CombatSituation`**.
- `Army В` у `Camp` / `Regrouping` → potential defender.
- `Army В`, яка до цього моменту вже увійшла в `Region` з метою `Camp` → potential defender; до `Battle Start` вона гарантовано буде в `Camp`.
- `Army В` у `Transit` або та, що вже вийшла з `Camp` → не defender.
- `Army В`, яка входить у `Region` після виникнення `CombatSituation` → не може долучитися до цього бою незалежно від своєї мети.

Протягом `Dt` В може для eligible `Army` вибрати: залишити для бою або `Retreat`. `Retreat` фактично виконується за **результатом/завершенням `CombatSituation`**, а не в момент натискання: втрат немає, після `Retreat` застосовується `Regrouping`. Для всіх `Army`, залишених у бою, В задає **один спільний `Defense Loss Threshold`**.

Після виникнення `CombatSituation` `Army В`, що була в `Camp` і потрапила до potential defenders, блокується для звичайного `Movement` до вирішення `CombatSituation`. Тобто не можна обійти бій командою «піти»; доступні лише `participation` або `pre-battle Retreat`.

Якщо В нічого не вибрав до Battle Start, тут логічно застосовується вже визначене нами загальне default-правило: якщо Defense Loss Threshold 0% — eligible Army відступають без втрат із наступним Regrouping; якщо >0 — беруть участь у бою.

І ці самі правила формування defender side та pre-battle decisions застосовуємо, коли атакують CampInRegion окупанта в чужій Owned Region — незалежно від того, атакує формальний owner чи третій Player.

Тут бачу одну важливу архітектурну зміну: CombatSituation.defenders[] краще розуміти не як «Army, які вже точно беруть участь у Battle», а як **зафіксований набір potential defenders**. Уже на Battle Start із нього після retreat decisions формується фактичний склад Battle. Це добре вкладається в поточну модель.

відступити повністю, або окремо для частини армії - тут була одруківка, мало бути "окремо для частини армій". Ми не дозволяєм гравцю перегруповувати армії або змінювати командувачів при бойовій ситуації

Так, зрозуміло. Тоді виправляємо трактування:

Гравець В може:

- наказати відступити **всім своїм eligible Army**;
- або вибрати **окремі Army**, які відступлять, а решта залишаться для бою.

Сам склад кожної Army при цьому незмінний. Після виникнення CombatSituation і до її завершення для potential defenders заборонені:

- merge/split Army;
- передача Soldier між Army;
- зміна Commander;
- звичайний Movement.

Доступне лише рішення для кожної Army: **залишитися для бою / відступити**. Для всіх Army, що залишаються, задається один спільний Defense Loss Threshold .

Regrouping Army при цьому теж є potential defender на тих самих умовах, оскільки ми домовились вважати її звичайною Camp Army для всього, крім ініціації руху та атаки.

В області виникає бойова ситуація А, але в ній уже є бойова ситуація В (окрім випадків незалежних бойових ситуацій). Ситуація А "стає в чергу". При цьому:

- не фіксується список потенційних захисників
- не фіксується локальна мета атакуючої армії (агресивний або не агресивний транзит) - вона буде визначена після попереднього бою за актуальним станом в області
- атакуюча армія переходить в спеціальний стан "очікування бою", в тому числі, якщо вона просто йшла транзитом, її транзит може бути затриманий через очікування бою. В цей час вона не може отримувати нові команди та ставати учасником інших бойових ситуацій.

Після бою або дострокового закінчення ситуації В відбувається ініціалізація ситуації А (фіксація локальної мети атакуючого, потенційних захисників) - цей момент вважається її початком. Привила участі захисників розраховуються відповідно їх стану на момент початку бойової ситуації. (У всіх описаних вище привалах замість "на момент входу атакуючого" читати "на момент початку бойової ситуації").

Якщо виникає що одна бойова ситуація - вона стає в чергу за А. Пріоритетів немає, всі ситуації розв'язуються в порядку виникнення.

Тобто, в подальшому треба розрізняти терміни:

[реєстрація бойової ситуації] - момент, коли відбувається одна з наступних подій:

- армія гравця А входить на територію гравця В (незалежно від наявності там його військ чи військ третіх гравців)
- армія гравця А входить на свою територію, окуповану іншим гравцем
- область гравця А стає окупованою, в той час, коли по ній уже рухається армія гравця А, яка увійшла туди до окупації, але запізно для того, щоб приєднатись до захисту
- гравці А та В стоять в кемпі в одній області, А ініціює атаку на В

[Початок бойової ситуації] - момент виникнення бойової ситуації, якщо немає конкуруючої бойової ситуація, або момент закінчення попередньої бойової ситуації (може бути дострокове

закінчення).

На етапі реєстрації бойової ситуації ми фіксуємо тільки атакуючу армію. Все інше фіксується при початку бойової ситуації. В тому числі, не фіксується не тільки склад армій захисту, а й гравець, який буде захищатися.

Може статися, що на момент початку бойової ситуації в області не виявиться інших гравців, або області стане нейтральною. тоді БС просто видаляється (але не раніше свого початку), армія продовжує рух попереднім маршрутом.

[Активна бойова ситуація] - ситуація, в якій вже наступив початок до її закінчення.

Атака на нейтралів або місто відбувається миттєво і не ставиться в чергу бойових ситуацій. Але, гравець не може атакувати нейтралів або місто, якщо він вже є потенційним учасником бойової ситуації в цій області. (захисником в активній бойовій ситуації, або атакуючою стороною в будь-якій)

Не повторюй, коментуй тільки по ділу

Є кілька важливих наслідків, але суперечностей з архітектурою не бачу.

1. `CombatSituation` фактично отримує чіткий lifecycle:

`registered` → `active` → `resolved`

`registered` — це саме черга. До переходу в `active` більшість бойових параметрів ще не існує. Це краще, ніж намагатися створювати повноцінну `CombatSituation` і потім переобчислювати її.

2. Локальна мета `Army` справді не повинна бути властивістю `queued CombatSituation`. Але я б не втрачала початковий маршрут/наказ `Army`. При активації ситуації локальна мета (`Camp` , `Transit` , `Aggressive Transit`) визначається заново за актуальним станом `Region`, але на основі все ще існуючого `Movement/order`. Інакше після скасування БС ми не знатимемо, який рух треба продовжити.

3. Виникає новий стан `Army`, і його краще зробити явним, наприклад `CombatWaiting`. Це не `Movement` і не `Camp`: просторово `Army` знаходиться в `Region`, але її `Movement` призупинений.

При видаленні БС без бою вона повертається до попереднього Movement і продовжує маршрут.

4. Реєстрація через **Occupation**, що виникла навколо вже присутньої **Army owner**, потребуватиме окремої події. Це важливо: тут немає `Movement.region_entry`, який природно зареєстрував би `CombatSituation`. Отже `Region.set_occupation()` (або відповідний метод) повинен знайти такі **Army owner** і зареєструвати для них БС.
5. Я б не зберігав `defender_player` взагалі до `start()`. При `start()` `CombatSituation` дивиться на актуальний стан `Region` і вже тоді або визначає `opponent/defenders`, або завершується без бою. Це добре вирішує зміну `owner/occupier` під час черги.
6. Потрібно буде акуратно розділити «БС зареєстрована» і «**Army є potential defender**». Заборона атакувати `City/Neutral Defense` поширюється на `attacker` будь-якої `registered` БС, але на `defender` — тільки активної. Отже простого `army.combat_situation != null` для всіх перевірок уже недостатньо.

Є одне питання, яке я б залишила на формалізацію черги: якщо одна й та сама **Army** за однією подією теоретично задовольняє кілька причин реєстрації БС, очевидно, треба не створювати дублікати. Найпростіший `invariant`: **Army може бути attacker максимум однієї незавершеної CombatSituation**. Це також прямо впливає з введеного тобою `CombatWaiting`.

2. Мета кемп або транзит - фіксована, залижить від початкового завдання армії. Тут йдеться про агресивний або неагресивний транзит - це визначається на момент початку бойової ситуації. Агресивний транзит виникає коли виконуються всі умови:
 - фінальна область маршруту не нейтральна, в ній на даний момент є війська гравця A, або вона належить гравцю A і не окупована
 - в поточна області не нейтральна, в ній є війська гравця A
5. Я так і сказав
6. Немає `potential defender` до початку БС

Так, прийнято.

По п.2 тоді в `Movement` треба чітко розділяти дві речі:

- `destination_mode : Camp | Transit` — визначається початковим завданням **Army** і під час очікування БС не змінюється.

- `transit_mode` : Normal | Aggressive — має сенс лише при `destination_mode = Transit` і визначається при початку `CombatSituation`, а не при її реєстрації.

Причому `Aggressive Transit` визначається саме за наведеними тобою двома умовами щодо фінальної та поточної `Region`. Якщо вони не виконані — `Transit` неагресивний.

По п.6 — так, моя фраза була неточною. До `CombatSituation.start()` **potential defenders** взагалі не існують. На `registered` stage відомий тільки `attacker`. Відповідно жодна `Army` не отримує обмежень через те, що вона *може стати* `defender` `queued` БС.

Це робить межу між `registered` та `active` досить чистою.

І на всякий випадок - якщо 2 армії одного гравця послідовно входять в чужу область - вони стають атакуючими 2 окремих БС в черзі. В будь-якій БС атакує тільки одна армія

Так. Це важливий invariant для `CombatSituation` :

одна `CombatSituation` = рівно одна `attacking Army`.

Тому дві `Army` A, що послідовно входять у `Region` B, реєструють дві окремі `CombatSituation`. Якщо перша вже `active/queued`, друга стає наступною в FIFO-черзі.

Армії одного `Player` не об'єднуються в `attacking side` однієї БС і не можуть виступати `reinforcement` для своєї `attacking Army`.

При цьому на стороні `defender`, навпаки, одна активна БС може включати декілька `Army` захисника — за погодженими правилами їх стану на момент `CombatSituation.start()` .

тепер опиши це чітко в новому одному файлі, починаючи від оновлених умов участі захисників в бою і до черги БС (правила, а не деталі реалізації)

Worked for 50s

Створив `docs/combat_situation_rules_update.md` .

Там окремо й послідовно описані:

- оновлені правила участі `defenders`;
- `default behavior` без ручних рішень;
- `Neutral Camp combat`;
- `attack` на `occupier`;

- CombatSituation Registration / Start / Active ;
- aggressive vs non-aggressive Transit;
- стан очікування queued attacker;
- FIFO-черга CombatSituation;
- винятки для незалежних combat, Neutral Defense і City Raid;
- City Raid defeat → Regrouping у тому самому Camp.

Commit: ad016959f6fb9d40338bf9b617364a5fdad665bf .

тепер онови опис моделей

Worked for 5m 38s

Оновив docs/MODELS.md одним комітом.

Основне, що внесено:

- новий lifecycle CombatSituation : Registered → Active → Resolved ;
- FIFO-черга конкуруючих БС;
- одна атакуюча Army на одну БС;
- CombatWaiting ;
- defenders фіксуються тільки на Start БС;
- нові правила Transit / Aggressive Transit / allow_transit ;
- заборона reinforcement після Start;
- pre-battle retreat і command lock;
- Dt як єдина базова константа;
- Regrouping тепер повноцінна Camp-presence для Annexation/Founding тощо;
- Regrouping більше не блокує merge/split сам по собі;
- City Raid / Neutral Defense винесені з player-vs-player queue;
- City Raid defeat → Regrouping у поточному Camp;
- Movement оновлено під queue/CombatWaiting.

Commit: 82dcd850200bb0ef12f15b9b1411335b76d08aa5 .

Проаналізуй опис моделей

Worked for 58s

Загалом опис уже досить цілісний, але я бачу кілька місць, які треба виправити або формалізувати до переходу далі.

Найсерьозніша проблема — Region.active_competing_combat і next_queued_combat . Вони описані як одиничні характеристики Region, але в нейтральній області ми дозволяємо одночасно

кілька незалежних БС між різними парами гравців. Отже в Region може бути кілька Active БС і фактично кілька незалежних FIFO-черг. Поняття «поточна БС області» вже не існує. Тут краще прибрати ці дві computed characteristics і визначати через методи щось на кшталт `find_competing_active(combat)` / `find_next_competing(combat)`. Черга є відносно конкретної БС, а не глобально одна на Region.

Друга проблема — `CombatWaiting`. Зараз він є окремим `Army.state`, але Army може потрапити в нього з двох принципово різних місць:

- із `Movement`;
- із `Camp`, наприклад `explicit attack` у `Neutral Region`, який став у чергу.

Для першої правильно зберігати `movement Food semantics`. Але для другої Army фізично продовжує стояти в `Camp`, повинна споживати локальну `Food` і, судячи з наших правил, не повинна раптом переставати бути `Camp-presence` для `Occupation/Annexation/Founding` лише через очікування своєї атаки. Нинішнє `has_eligible_presence = state in {Camp, Regrouping}` таку Army виключить. Треба або не використовувати `CombatWaiting` як взаємовиключний `location state`, або додати поняття `underlying state/context` (`Camp` чи `Movement`) і всі `presence/food` правила будувати по ньому.

Третє — `attacker_destination_mode`. В правилах воно фіксоване початковим завданням Army, але ми також домовилися, що на `Registration` БС фактично фіксується тільки `attacker`, а решта бойового контексту визначається на `Start`. Зараз воно записане прямою характеристикою `CombatSituation` без чіткого моменту встановлення. Найчистіше: початковий `Camp/Transit task` зберігає `Movement`, а `CombatSituation.start()` читає його і тоді записує `attacker_destination_mode`. `Aggressive/NonAggressive` теж визначається на `Start`, як уже описано.

Четверте — визначення `defender_player` на `Start` поки занадто абстрактне. Особливо важливий випадок: `Region` формально належить `B`, але окупована `C`, і входить `A`. За нашими правилами актуальним противником може бути `occupier C`, а не `formal owner B`. Це варто прямо формалізувати в `start()`, інакше саме тут легко зробити неправильну реалізацію для `third-party attack` на `occupier`.

П'яте — `Neutral Defense attack` фактично випав із API моделей. Для `City` є `Army.user_raid_city()`, для `player attack` є `Army.user_attack_player()`, а симетричного `Army.user_attack_neutral_defense()` у поточному описі немає. Він потрібен: тільки конкретна Army в `Camp`, не `Regrouping`, і вона не може бути `attacker незавершеної БС` або `potential defender Active БС`.

Є ще одна концептуальна нечіткість: секція `CombatSituation` тепер прямо каже, що ця модель — `player-vs-player lifecycle`, але нижче в ній описані `Abstract defense combats`. Треба визначитися: `Neutral Defense/City Raid` теж створюють `CombatSituation`, тільки особливого `instant type`, чи вони взагалі не є `CombatSituation`, а лише використовують спільний `combat-calculation` механізм. Зараз текст допускає обидва трактування. Я б зробив друге: `CombatSituation` — тільки `PvP interaction`, а `нейтрали/City` — окремі одноразові `Army actions`, які викликають спільний `combat resolver`.

Ще два менші моменти. `registration_order` як глобальний `FIFO` номер `Region` допустимий, але при незалежних БС він задає лише порядок серед тих `situations`, які взаємно конфліктують — це варто сказати явно. І `defender_combat_threshold`: треба явно зазначити, чи `0%`, введені вручну у `dialog`, означають `Retreat` усіх `Army`, що ще залишені для бою. Зараз `0%` однозначно має це значення лише для `persistent threshold` при відсутності рішення.

В іншому останні зміни лягли нормально: `Regrouping` узгоджений з `Camp-presence`, `reinforcement` після `Start` прибраний, `potential defenders` фіксуються в правильний момент, `queue attacker` = одна `Army`, `Dt` використаний послідовно, а `City Raid defeat` тепер не конфліктує із загальним `Retreat`.

Я б у першу чергу виправив **модель черги та `CombatWaiting/Camp-presence`** — це вже не косметика, а речі, які можуть привести до неправильної структури моделей.

Yesterday 8:40 PM

Змінемо умову визначення агресивного транзиту.

Гравець відправляє армію за маршрутом, фінальна обсть якого `F`. В цей момент ми фіксуємо [цільового суперника] (ЦС):

- `F` нейтральна: цільового суперника немає
- `F` налижить гравцю `A`, не окупована: ЦС - `A` (в тому числі, якщо його військ в області `F` на даний момент немає)
- `F` налижить гравцю `A`, але окупована `B`: ЦС - `B`

Надалі, під час руху маршрутом транзит через області, що налажать ЦС і не окуповані - агресивний транзит. Транзит через області, що належить іншим гравцям - не агресивний.

ЦС фіксується при створенні маршруту і не змінюється, якщо ситуація в кінцевій області зміналася. Але, за бажанням, гравець може ініціювати оновлення ЦС відповідно до актуальної ситуації в області `F`.

Зміна вступить в силу при вході в наступну область на маршруті.

Правила виникнення бойової ситуації не змінюються, змінюється тільки умова агресивного транзиту.

ЦС також визначає, який поріг втрат використовується при атаці:

- бій з ЦМ в будь-якій області - основний поріг атаки
- бій з іншим гравцем в будь-якій області - поріг побічної атаки.

В тому числі, поріг побічної атаки застосовується в фінальній області, якщо в ній інший гравець.

Так, це чистіше за попереднє визначення агресивного транзиту. Воно прибирає залежність від поточної військової присутності в проміжній області.

Є одна описка: у пункті «F належить гравцю A» ти, очевидно, маєш на увазі власника F, а не гравця, який відправляє армію. Інакше ЦС виходить самим гравцем. Я б у формалізації назвав власника F, наприклад, B .

Логіка тоді така: `target_opponent` є характеристикою маршруту/`Movement`, а не `CombatSituation`. Він визначається при створенні маршруту за станом F: `Neutral → null` ; `Owned B, not Occupied → B`; `Owned B, Occupied C → C`.

Ручне оновлення ЦС переобчислює його за тим самим правилом, але нове значення застосовується лише починаючи з входу `Army` в наступну `Region`. Це важливо: воно не повинно змінювати `classification` уже зареєстрованої/активної БС у поточній `Region`.

При `Transit classification` на вході в `Region`:

- `Region` належить ЦС і не `Occupied` → `Aggressive` ;
- будь-який інший випадок → `NonAggressive` .

Тобто `Occupied Region` ЦС сама по собі **не дає `Aggressive Transit`**, бо умова прямо вимагає «належить ЦС і не окупована».

І для кожної `PvP CombatSituation` `attack threshold` тепер визначається дуже просто: фактичний `defender` на `Start` = ЦС → `target_combat_threshold` ; будь-який інший `defender` → `incidental_combat_threshold` . Це справді поширюється і на F: якщо за час походу там змінився противник, але гравець не оновив ЦС, бій із новим противником використовує `incidental threshold`.

Це також усуває одну з проблем поточного `MODELS.md` :

`CombatSituation.determine_transit_mode()` більше не повинна аналізувати `final/current Region` за

старою формулою. Classification походить від зафіксованого для цього відрізка руху ЦС.

по combat situation в нейтральній області - хай всі стають в чергу, не будемо окремо оброблювати "незалежні" (атака на нейтралів і місто все ще поза чергою)

Так. Це суттєво спрощує правило черги.

У межах однієї Region існує одна спільна FIFO-черга всіх player-vs-player CombatSituation, незалежно від власності Region і від того, які пари гравців беруть участь.

Отже, в Neutral Region навіть БС А↔В і С↔D не можуть бути Active одночасно. Друга чекає завершення першої, потім стартує й отримує свої повні Dt до Battle Start.

Винятки залишаються ті самі: атака Neutral Defense і City Raid — миттєві взаємодії поза чергою CombatSituation. Правила щодо заборони такої атаки для гравця, вже залученого в PvP CombatSituation, теж не змінюються.

Це заодно прибирає проблему, яку я щойно відзначав з active_competing_combat: тепер у Region справді може бути одна active_combat_situation і одна лінійна FIFO-черга.

по зауваженням до моделей ще треба моя участь?

Майже ні. Після двох останніх рішень із моїх зауважень залишилося одне, де потрібне твоє рішення:

CombatWaiting для Army, яка почала атаку з **Camp** і стала в чергу. Я вважаю, що вона повинна весь час очікування залишатися Camp-presence: продовжувати споживати місцеву Food, підтримувати Occupation/Annexation/Founding тощо. CombatWaiting тоді краще трактувати не як заміну фізичного стану Camp/Movement, а як окремий combat-lock/статус очікування.

Решта вже або визначена твоїми правилами, або технічна:

- ЦС та нова класифікація Aggressive Transit — визначили;
- єдина FIFO-черга на Region — визначили;
- визначення актуального defender на Start можна формалізувати з уже прийнятих правил;
- user_attack_neutral_defense() просто треба додати;
- Neutral Defense/City Raid можна однозначно винести за межі моделі CombatSituation.

Тобто достатньо вирішити питання CombatWaiting.

так, залишається в кемпі

Тоді всі принципові зауваження до моделей закриті. `CombatWaiting` не змінює фізичну присутність `Army`: якщо вона чекала БС із `Camp`, вона залишається `Camp-presence` з усіма відповідними наслідками.

внеси зміни в моделі

Worked for 2m 8s

Вніс зміни в `docs/MODELS.md`.

Оновлено:

- новий [цільовий суперник] / `target_opponent` ;
- його визначення при створенні маршруту;
- ручне оновлення ЦС із набуттям чинності при вході в наступну `Region`;
- нове правило `Aggressive Transit` ;
- вибір `target_combat_threshold` / `incidental_combat_threshold` через фактичного противника відносно ЦС;
- одна спільна FIFO-черга всіх `PvP CombatSituation` у `Region`, включно з `Neutral Region`;
- прибрані «незалежні» паралельні `PvP БС`;
- `CombatWaiting` більше не є фізичним `Army.state` : `Army` з `Camp` лишається `Camp-presence`, `Army` з `Movement` зберігає `Movement context`;
- додано `user_attack_neutral_defense()` ;
- `Neutral Defense attack` і `City Raid` явно винесені за межі `CombatSituation` .

Commit: `b88191e448442869670effc0bcc6a2e9b978103c` .

проаналізуй

Worked for 33s

Після останніх змін структура стала значно чистішою. Але знайшов одну явну помилку і кілька менших неузгодженостей.

1. Серйозна помилка в `Potential defenders` .

Зараз написано, що `Army`, яка до `Start` уже ввійшла для `Transit` , є `Not eligible` . Це суперечить узгодженому правилу: така `Transit Army` може вирішити зупинитися й брати участь у `defense`, якщо на `Start` уже є бойова ситуація, створена через `Camp/Regrouping/Camp-bound` війська, і вона ще не вийшла з `Region`.

Причому нижче залишився текст:

Transit Army не беруть участі й продовжують Transit

тобто сама структура документа натякає, що Transit Army мають бути серед тих, кому може бути доступне рішення, але вище вони повністю виключені з potential defenders.

Тут треба розділити:

- potential defenders у Camp/Regrouping/Camp-bound — автоматично належать до defensive context;
 - Transit Army, що ввійшли до Start і ще фізично не вийшли, — можуть добровільно перейти до участі в цій БС;
 - Transit Army після Start — вже ні.
2. У кількох місцях залишилася стара термінологія competing CombatSituation .

Наприклад is_queued :

```
status == Registered і існує earlier competing CombatSituation
```

і Start:

якщо немає конкуруючої CombatSituation .

Після останнього рішення ніякої класифікації «конкуруюча / незалежна» більше немає. В Region одна FIFO-черга всіх PvP БС. Це треба просто прибрати, щоб реалізація не почала шукати якийсь is_competing() .

3. CombatSituation.transit_mode зараз описаний як пряма характеристика, але фактично він уже повністю детермінований target_opponent snapshot + Region ownership/occupation .

Це не обов'язково помилка — його можна зберігати як результат Start. Але треба чітко сказати, що він фіксується на Start , а не на Registration. Зараз опис характеристики може читатися так, ніби значення вже існує одразу.

4. З ЦС є одна архітектурна деталь: pending_target_opponent не може сам по собі представляти три стани:
- нема pending update / pending Player / pending null .

У тексті це вже помічено словами, але моделі бракує самої характеристики-флага. Треба або:

target_opponent_update_pending: bool + pending_target_opponent ,

або value-object/optional wrapper. Інакше характеристика описана логічно, але структура моделі неповна.

5. Є потенційно невизначена взаємодія, де вже може знадобитися твоє рішення: `queued/Active attacker`, який фізично лишається в `Camp`, продовжує бути `Camp-presence` для `Annexation/Founding`. Це ми визначили. Але не визначено, чи може через таку `Army` **завершитися Annexation або Castle Founding під час її combat lock**. Сам `lock` забороняє змінювати `Army`, але `Annexation` змінює `Region` і може повністю змінити сенс БС, яка стоїть у черзі.

Продовження `progress` тут однозначно правильне за нашими правилами `Camp-presence`. А от ручний `user_annex_region()` / завершення `Founding` під час `Active` або `queued` БС варто окремо визначити.

І ще дрібне: в `Army.*movement` досі є фраза «при `CombatWaiting` », хоча `CombatWaiting` як `state` ми щойно прибрати. Треба замінити на `is_combat_waiting` / `queued CombatSituation`.

Тобто перед наступним редагуванням від тебе потрібне фактично лише рішення по пункту 5. Решту можна виправити механічно.

що таке `eligible` ?

`eligible` тут означає «такий, що має право / відповідає умовам для участі».

Наприклад `eligible defender` = `Army`, яка за правилами може брати участь у цьому бою як захисник.

У нашому описі краще не зловживати цим словом. Для `CombatSituation` точніше використовувати конкретні терміни:

- `potential defender` — потенційний захисник;
- `participant` — фактичний учасник бою.

Тоді не виникатиме третього нечіткого поняття `eligible` .

1. - невизначеність в правилах, чи в описі моделей?

В описі моделей. Самі правила тут уже визначені.

Проблема в тому, що `MODELS.md` зараз **неправильно відображає прийняте правило**: він безумовно виключає `Transit Army`, яка увійшла до `Start`, тоді як за нашими правилами вона може вирішити залишитися для бою, якщо активна БС виникла з іншої причини і `Army` ще не вийшла з `Region`.

Тобто твого рішення по п.1 не потрібно — це треба просто виправити в моделях.

4 - моє рішення треба?

Ні. Пункт 4 — суто технічне представлення трьох можливих станів:

1. оновлення ЦС не запитувалось;
2. запитано новий конкретний Player;
3. запитано змінити ЦС на `null`.

Це можна однозначно реалізувати в моделях без зміни ігрових правил. Твого рішення не потрібно.

На всякий випадок уточню. Гравець натискає кнопку "Оновити ЦС" - новий ЦС фіксується в цей момент, але зміна набуває чинності тільки з наступної області (новий ЦС не обирається в момент входу в нову область).

Так. Це важливе уточнення, і воно прибирає можливу двозначність.

У момент натискання «Оновити ЦС» система **одразу обчислює і фіксує конкретне нове значення ЦС** за поточним станом кінцевої області `F`. Воно зберігається як `pending_target_opponent`.

При вході в наступну Region нічого заново не обчислюється — просто:

```
target_opponent = pending_target_opponent
```

і `pending`-значення очищується.

Тобто якщо між натисканням кнопки та входом у наступну область ситуація в `F` зміниться, це **не впливає** на вже зафіксований новий ЦС.

У моделях зараз це майже так і написано, але я б при виправленні п.4 сформулював це явно.

5. Якщо в області є бойова ситуація - значить там є армії різних гравців, значить обидва таймер приєднання призупинений і приєднувати не можна. Це ж стосується заснування замку

Так. Тоді пункт 5 теж закритий.

Треба зафіксувати загальніше: якщо в Region є активна/зареєстрована бойова ситуація через присутність різних Player, прогрес Annexation і Castle Founding призупиняється, а завершити/виконати приєднання в цей час не можна.

Отже, додаткового рішення від тебе для виправлення моделей уже не потрібно.

призупиняється зупиняється не через бойову ситуацію, а через сам факт присутності інших армій

Так, це точніше. Причина не `CombatSituation` .

`Annexation` і `Castle Founding` призупиняються через **blocking foreign presence** у `Region`. Бойова ситуація лише часто є наслідком тієї самої присутності.

Отже, у моделях не треба вводити залежність `can_progress` від наявності `CombatSituation` .
Поточну логіку через `foreign presence` треба зберегти/уточнити.

оновлюй моделі

Worked for 1m 46s

Оновив `docs/MODELS.md` .

Виправив:

- `Transit defender` до `Start`: може добровільно залишитися для бою, але сам по собі не створює `battle context`;
- прибрав залишки `competing CombatSituation` ;
- `transit_mode` тепер явно визначається на `Start`;
- `pending_target_opponent` фіксується саме в момент натискання «Оновити ЦС», без переобчислення при вході в наступну `Region`;
- додав окремий `has_pending_target_opponent_update` , щоб відрізнити `pending null` від відсутності оновлення;
- один спільний `defender threshold` для всіх `Army`, що беруть участь у `defense`;
- заборону `Neutral Defense / City attack` привів до `Player-level`;
- `Annexation` і `Castle Founding` призупиняються саме через `foreign Army presence`, а не через `CombatSituation` ;
- прибрав залишок старого `CombatWaiting` як фізичного `state`.

Commit: `ea26c770b134f8a5313e307302ac3b19f6675d21` .

аналізуй

Worked for 58s

Після повторного проходу бачу таке.

1. Є проста помилка редагування: у `CampInRegion` рядок `can_progress` продубльований двічі.
Це просто прибрати.

2. Для Transit-захисника правило вже описане правильно, але модель не дає його реалізувати. Ми написали, що Army, яка ввійшла для Transit до Start, може вирішити залишитися для бою, але:
 - її немає в `potential_defenders[]` ;
 - немає окремого decision на кшталт `join_defense` ;
 - немає method для цього рішення.

Треба додати окремий список/decision для таких pre-Start Transit Army або включати їх у ширший список `potential_defenders` , позначаючи, що за замовчуванням вони продовжують Transit. Це технічне питання, твого рішення не потребує.

3. `defender_combat_threshold` зараз спільний — це виправлено. Але лишилась невизначеність: яке значення використовується, якщо Player взагалі не задав `threshold` у діалозі. Старе правило через `persistent threshold` кожної Army ми прибрали, і правильно, але нового default для єдиного `threshold` у моделях немає. Тут уже потрібне твоє рішення.
4. Є ще одна справжня невизначеність правил: `explicit attack` із Camp у Neutral Region. Наприклад А натиснув «атакувати В», БС стала в чергу. На Registration ми принципово не фіксуємо `defender_player` . Коли через деякий час ця БС Start-ує, В вже може піти, а в області може залишитися С.

Потрібно визначити: ця атака тоді

- зникає, бо ціллю був саме В;
- чи на Start може перетворитися на атаку проти іншого актуального Player.

Зараз модель формально дозволяє другий варіант, але я не вважаю, що це можна вирішити без тебе.

5. Для Annexation/Founding варто лише уточнити термін. Зараз написано `blocking foreign Army presence` . За вже прийнятими правилами це має означати саме нашу ігрову «присутність»: Camp, Regrouping, Army що вже ввійшла і йде до Camp; звичайний Transit, що проходить область, не блокує. Інакше фраза `foreign Army presence` може бути прочитана занадто широко.
6. Дрібна термінологічна чистка: у `Knight.user_change_unit_composition()` і `Army.user_change_commander()` ще трапляється формулювання про lock «активної CombatSituation», хоча `queued attacker` теж `locked`. Методи фактично вже спираються на `is_command_locked` , тому це лише текст.

Отже, від тебе зараз потрібні рішення лише по №3 і №4. Решта механічно виправляється.

4. Для такої атаки `defender player` треба зафіксувати. Якщо на момент start бойова ситуація виявилась не актуальна - вона просто видаляється. Можна розділити модель `CombatSituation` на декілька підмоделей по типам ситуації.

Базова моделі з спільними характеристиками й методами і декілька моделей для окремих типів її наслідують

Так, це логічніше.

Для explicit attack із Camp у Neutral Region defender_player треба фіксувати вже на Registration, бо Player явно обирає конкретну ціль. Якщо на Start у B вже немає релевантної присутності і interaction більше не може відбутися — ця CombatSituation завершується без бою. Вона не повинна «перенацілюватися» на C.

Ідея розділити CombatSituation на базову модель + спеціалізовані підтипи тут доречна. У нас уже є різні сценарії з різними правилами визначення defender-a:

- вторгнення в чужу Owned Region;
- вхід owner-a у власну Occupied Region;
- late owner Army після виникнення Occupation;
- explicit attack конкретного Player у Neutral Region.

Базова CombatSituation могла б містити спільне: region , attacker , status , registration_order , start_time , queue lifecycle, potential_defenders , decisions, Battle Start через Dt , resolve/finish.

А підтипи визначали б:

- як фіксується або визначається defender_player ;
- коли situation вважається актуальною на Start;
- звідки береться combat mode / attack-defense strength semantics;
- специфічні registration data.

Для NeutralCampAttackCombatSituation я б однозначно зберігав target_player / defender_player на Registration і не переобчислював його на Start.

Це також прибере нинішню нечіткість базового start(): determine defender Player , бо для різних типів це реально різні алгоритми.

Але по розділенню не впевнений, доведеться уточнювати архітектуру, наслідування характеристик. Можливо простіше вставити всю логіку для різних випадків в одну модель

Я б поки залишив **одну модель CombatSituation** .

Різниця між нашими типами БС поки недостатньо велика, щоб виправдовувати наслідування. Основний lifecycle у них однаковий: Registration → queue → Start → Dt → Battle/finish. Відрізняється переважно логіка кількох полів залежно від `registration_reason`.

Можна зробити `registration_reason` фактично discriminator-ом і мати всередині методи на кшталт `resolve_defender_player()` та `is_still_valid_at_start()`, які розгалужуються за типом ситуації.

Для explicit Neutral Camp attack особливість буде проста: `defender_player` фіксується на Registration; на Start лише перевіряється, чи атака на цього Player усе ще актуальна. Для територіальних випадків defender, як і раніше, визначається на Start за актуальним ownership/occupation.

Якщо пізніше відмінностей стане багато — тоді вже буде аргумент для підмоделей. Зараз наслідування скоріше ускладнить опис моделей і майбутню архітектуру без помітної користі.

5. Все правильно - кемп та вхід для кемпу блокують, транзит не блокує. Це створює трохи проблем для випадку створення замку на своїй області, бо тоді при транзиті чужої армії бойова ситуація є, а призупинення будівництва немає. Поки залишим це як є.

Вноси зміни

Worked for 1m 24s

Вніс зміни в `docs/MODELS.md`.

Зафіксовано:

- explicit Neutral Camp attack фіксує конкретного `defender_player` на Registration;
- на Start така БС або лишається атакою саме цього Player, або видаляється без бою; перенаправлення на іншого Player немає;
- `CombatSituation` поки лишається однією моделлю, різниця типів описується через `registration_reason`;
- pre-Start Transit defender оформлений окремо як candidate з явним рішенням приєднатися до defense;
- Annexation/Founding блокує лише foreign Camp/Regrouping або entered-for-Camp presence; Transit не блокує навіть за наявності `CombatSituation`;
- виправив формулювання `command-lock` для Commander/unit composition.

Пункт із `default defender_combat_threshold` , якщо Player нічого не задав, не чіпав — він поки лишається невизначеним.

Commit: 1814cb7e9172705083fb3d49ef22af8d63fe9d74 .

аналізуй

Worked for 26s

Після останніх змін бачу 5 моментів.

1. Є пряма суперечність у `Queue` : зараз написано, що `queued situation` «не має `defender Player`». Це вже неправильно для `explicit Neutral Camp attack` — там `defender_player` фіксується ще на `Registration`. Треба уточнити: для територіальних `queued BC defender` ще не визначений, для `explicit attack` — уже визначений.
2. `start()` теж залишився зі старим універсальним формулюванням «визначає актуального `defender Player`». Для `explicit attack` він не повинен його визначати, лише перевірити актуальність уже зафіксованого.
3. `Transit defender` після `user_join_defense_from_transit()` має логічну дірку. Написано, що він стає `participating defender` і отримує `command-lock`, але `Army.active_defender_combat_situation` та `is_command_locked` зараз зав'язані на входження в `potential_defenders[]` . Найпростіше: при `join` переносити `Army` з `transit_defender_candidates[]` у `potential_defenders[]` . Тоді вся вже існуюча `lock-логіка` працює без нового поняття `participating_defenders` .
4. У `CastleFounding` випадково двічі продубльований `cancel()` .
5. Залишається раніше невирішене питання `default defender_combat_threshold` . Зараз є два назви — `direct defender_combat_threshold` і `computed defender_loss_threshold` , але якщо Player нічого не встановив, значення не визначене. Це вже не просто текстова помилка — без правила `Battle Start` не може однозначно зафіксувати `threshold`.

Ще одна дрібниця: для `explicit Neutral Camp attack` фраза «interaction з В досі актуальний» поки не формалізована. Я б без додаткового рішення трактував це як: у В на `Start` є хоча б одна допустима для `defense Camp/Regrouping/entered-for-Camp Army`. `Transit` сам по собі не робить цю `explicit атаку` актуальною. Це узгоджується з уже прийнятою логікою `Transit`.

5. Всі пороги встановлюються при формуванні армій або відпраці. Деталі визначим про розробці UI, але на момент `CombatSituation` всі пороги будуть встановлені. Виправляй

Worked for 1m 35s

Виправив моделі.

Зміни:

- `queued explicit Neutral Camp attack` тепер може вже мати `defender_player` ; `territorial` — ні;
- `start()` розділяє ці випадки;
- `Transit defender` при `join defense` переноситься в `potential_defenders[]` , тому стандартний `defender lock` працює без окремої логіки;
- для `explicit Neutral Camp attack` формалізована актуальність на `Start`: потрібна `Camp/Regrouping/entered-for-Camp Army` зафіксованого `defender-a`; чистий `Transit` не достатній;
- `defender threshold` більше не має `default/fallback` у `CombatSituation` : вважається вже встановленим до неї під час формування/відправлення `Army` або іншої попередньої UI-дії;
- `lock_combat_parameters()` лише фіксує вже визначені пороги.

Commit: 7d8e7f6dd10cfcd63e3fe0257577c603ba4307 .

відкладемо поки ці ітерації і обговоримо деякі загальні питання.

1. Може краще буде відмовитись від обмеження на використання ланцюгу посилань при обчислюванні обчислювальних характеристик. Натомість, якщо х-ка моделі А використовує `A->B->...->C-some_param` то в описі зв'язків між моделями прямо задавати, що А залежить від С ? Щоб не переобтягувати проміжні моделями х-ками, які потрібні для інших моделей. Багато таких у нас зараз?

Worked for 12s

Так, я схильна до того, що це буде **краще правило**, ніж нинішнє обмеження ланцюжків.

Зараз заборона `A → B → C.param` змушує нас створювати в В проміжну `computed`-характеристику навіть тоді, коли для самої В вона концептуально нічого не означає. Це поступово засмічує моделі API-полями, єдина причина існування яких — дати іншій моделі коротший шлях.

У поточному `MODELS.md` таких випадків уже помітно кілька. Найочевидніші:

- `Knight.current_region = army.current_region` і `Knight.current_camp = army.camp` . Вони значною мірою є проксі до `Army` , щоб `Knight` не ходив далі по зв'язках.
- `Castle.castle_region_army_food_consumption = region.camp_food_consumption` . Тут `Castle` фактично залежить від присутності `Army/Camp` у своїй `Region`, але залежність протягнута через `Region`.
- `Recruitment.can_progress` використовує `castle.player.empty_coins` . Це вже буквально `Recruitment → Castle → Player` .

- `Castle.can_annex_region()` прямо обходить `CampInRegion` → `armies[]` → `knights[]`. Тобто для `methods` ми фактично вже дозволили саме те, що забороняємо для `computed characteristics`.
- `Army.is_combat_waiting` і `active_defender_combat_situation` залежать від `CombatSituation` через `reverse relationships`; тут теж `dependency graph` важливіший за довжину шляху.

Але я б не формулював нове правило як просто «ланцюжки дозволені без обмежень». Краще:

`Computed characteristic` може читати характеристики через довільний `relationship path`. При цьому її `dependency metadata` повинна містити **всі моделі/характеристики, зміна яких може змінити результат**, незалежно від того, через які проміжні `relationships` вони досягаються.

Тобто якщо `A.x = A.b.c.some_param`, у `dependency graph` явно є залежність `A.x ← C.some_param` (і, важливо, залежності від `relationships` `A→B` та `B→C`, бо заміна самого `B` або `C` теж змінює результат).

Це дає нам дві переваги: моделі описують **предметну область**, а не технічні проксі для `dependency engine`; водночас `infrastructure` все одно точно знає, що треба перерахувати при зміні `C`.

Я б навіть пішов далі: після прийняття цього правила варто пройти `MODELS.md` і видалити **лише ті `computed`-характеристики, які виявляються чистими проксі**. Не треба прибирати

`Knight.current_region`, наприклад, автоматично: якщо `current_region` є корисним поняттям саме для `Knight` і багато його правил ним оперують, воно може залишитися як нормальна `derived characteristic`. Критерій має бути семантичний, а не довжина ланцюжка.

Тобто проблема вже є, але поки не критична. Саме зараз хороший момент змінити правило, поки таких проксі не накопичилось значно більше.

поки не приймаємо, ще подумаю.

Вже думаю про перехід до `front-endy`. Ми говорили про підписки `fron-end-y` на оновлення моделей. Такий варіант:

на бек-енді до основних моделей створюємо спеціальні моделі - посередники між моделями бек-енд логіки та фронт-ендом. Цю моделі будуть збирати інформацію, доступну гравцям, контролювати її і будуть відслідковувати оновлення.

Наприклад, модель `UserRegion` - існує, поки гравець на фронт-енді переглядає карту з видимою цією областю, збирає з основних моделей інформацію про область, доступну гравцю, відслідковує оновлення.

Так, як напрям це виглядає сильніше, ніж давати frontend-у прямі підписки на domain models. По суті це окремий **projection/read-model layer**, причому персоналізований під конкретного Player.

UserRegion у такій схемі може бути не копією Region, а саме представленням області для конкретного користувача: збирати Region, City, CampInRegion, Army, movement/combat information тощо; застосовувати visibility rules; віддавати frontend-у вже готовий стан; генерувати update, коли змінилася будь-яка залежність, що впливає на видимий результат.

Ідея «існує, поки область реально потрібна frontend-у» теж хороша, але я б поки не робив саме **час життя об'єкта** частиною gameplay/domain-моделі. Це радше runtime subscription object:

Frontend subscription → UserRegion projection → domain models

Коли клієнт показує ділянку карти, backend створює/активує потрібні UserRegion(player, region) projections. Коли ділянка більше не потрібна — subscription можна прибрати, а projection знищити або залишити кешованою. Це вже технічне рішення.

Головна перевага — дуже чітка межа безпеки. Frontend взагалі не повинен знати, що в Region є приховані Army або які в них точні параметри. Він отримує лише те, що UserRegion дозволив конкретному Player. Тобто контроль доступності інформації централізується на backend.

Є ще важливий плюс для нашої trigger/computed архітектури: UserRegion може мати явні dependencies на domain models і реагувати на їх зміни так само, як інші залежні обчислення. Тоді frontend не мусить розуміти, що зміна Army, CombatSituation чи City означає необхідність оновити область — це знає projection layer.

Я б лише не називав їх просто «моделями» нарівні з Region, Army, Castle. Архітектурно це інший клас сутностей. Умовно:

Domain Model → User Projection → Frontend

І UserRegion — цілком нормальна назва для конкретної projection. Аналогічно потенційно з'являться UserArmy, UserCastle, UserCombatSituation тощо, але створювати їх наперед не треба: частину інформації можна віддати вкладеною в UserRegion.

Є одне питання, яке тут стане принциповим трохи пізніше: чи UserRegion зберігає власний стан/останню відому гравцеві інформацію, чи завжди є чистою projection поточного domain state. Це напряму пов'язано з fog of war та «останньою відомою інформацією». Поки це можна не вирішувати.

Другий варіант - як планувалося раніше: не створювати окремі моделі, мати характеристики для фронт-енду в основних моделях і фронт-енд сигнатури для відстеження оновлення. Що думає, який краще?

Я б для нашої гри вибрав **перший варіант** — **окремий projection layer (UserRegion тощо)**. Не через сам frontend, а через те, що в нас інформація для гравця вже суттєво відрізняється від повного стану backend.

У другого варіанта є сильна перевага — він простіший. Наприклад, Region має frontend_* характеристики/сигнатури, frontend підписується на Region, backend при зміні залежностей повідомляє клієнт. Для першої версії це менше коду і менше сутностей.

Але далі виникає фундаментальна проблема: **стан Region один, а представлення Region різне для різних Player**. Один бачить свою армію повністю, інший — лише факт її присутності, третій, можливо, взагалі її не бачить. Те саме з напрямком руху, CombatSituation, складом Army тощо. Тоді основна Region починає обростати не просто frontend-характеристиками, а логікою виду «що бачить Player X».

3 UserRegion(player, region) це природно:

Region + Army + Camps + City + CombatSituation + visibility → UserRegion(player) → frontend

І особливо важливо: підписка стає підпискою не просто на **зміни даних**, а на **зміни того, що доступно конкретному гравцю**. Наприклад, чужа Army не змінилася взагалі, але стала видимою Player A. UserRegion(A, R) змінився, хоча Army — ні. У другому варіанті це доведеться якось впишувати в сигнатури domain models.

Є й архітектурний плюс: domain model тоді взагалі не знає про frontend. Army, Region, CombatSituation описують гру. UserRegion описує інформацію, яку система показує конкретному користувачу. Це хороше розділення відповідальностей.

Але я б трохи змінив твою першу ідею: **не створював би дзеркальну User-модель для кожної domain model**. Тобто не робити автоматично UserRegion, UserArmy, UserKnight, UserCity, UserCamp... Це швидко перетвориться на другий паралельний object graph.

Краще створювати projections відповідно до потреб UI. Наприклад, для карти UserRegion може одразу містити все необхідне для відображення hex: owner, city, доступні ресурси, видимі війська,

рухи, combat indicators тощо. Для вікна власної Army може існувати окрема `UserArmyDetails`. Для замку — `UserCastleDetails`.

Тобто я б заклад архітектуру:

Domain models → player-specific projections → subscriptions → frontend.

А не:

Domain models → frontend fields/signatures → frontend.

Другий варіант я б обрав лише для гри, де майже весь серверний стан однаково доступний усім гравцям. У нас це, схоже, буде не так.

логічно. Приймаєм `player-projections`, деталі обговорим пізніше

Прийнято. Фіксуємо архітектурний напрям: між `domain models` і `frontend` буде окремий шар **player-specific projections**. Конкретний склад `projections`, їх `lifecycle`, `dependencies`, `subscriptions` і формат оновлень визначимо пізніше.

підозрюю, при оновленні опису моделей ти не тільки виправляєш знайдені раніше проблеми, а повністю переписуєш файл, через що з'являються нові проблеми. Так?

Не зовсім. Я не генерую `MODELS.md` заново з нуля при кожній зміні.

Зазвичай я беру поточний файл із GitHub і роблю текстові заміни конкретних фрагментів, після чого записую назад **весь отриманий вміст файлу** через GitHub API. Тобто технічно весь файл передається при `commit`, але незмінені частини мають залишатися байт-в-байт тими самими.

Проте твою підозру варто перевірити по `git diff`: якщо в попередніх комітах змінювалися фрагменти, яких завдання не стосувалося, значить у моєму процесі справді були небажані побічні правки. І це може пояснити частину нових проблем.

Надалі правильне правило для мене: **перед commit перевіряти diff і не допускати жодних змін поза явно потрібними фрагментами**. Це значно надійніше, ніж покладатися лише на те, що операції заміни були локальними.

наче зайвих змін не видно в останньому коміті. Вноси нові правки в опис моделей. Принцип невикористання ланцюгів зв'язків поки залишається, проміжні характеристики залишаються.

Worked for 1m 7s

Вніс у docs/MODELS.md .

Додав тільки зафіксований принцип окремого шару `player-specific projections` між backend domain models і frontend. Деталі моделей `projections`, `lifecycle`, `dependencies` та формату підписок залишив невизначеними до окремого обговорення.

Правило невикористання ланцюгів зв'язків для `computed characteristics` не чіпав, проміжні характеристики залишились.

Commit: `a783fd094fca87383b6e27dac0feb2943f398627`

Перевірив diff: лише 6 доданих рядків, жодних інших змін.

аналізуй опис моделей і зразу вноси необхідні правки, якщо вони не вимагають моєї участі

Worked for 11m 34s

Проаналізував `MODELS.md` і одразу виправив усе, що можна було виправити без нового рішення з твого боку.

Основні правки: прибрав кілька фактичних порушень правила без ланцюгів зв'язків через проміжні `player_id` , `player_empty_coins` , `Knight.player` , `Army.is_entered_for_camp` ; додав `Region.blocking_camp_presence_player_ids[]` ; виправив `Recruitment` ; узгодив `register_combat()` з `explicit Neutral Camp attack`; прибрав суперечність із «видаленням» `CombatSituation` поза `lifecycle Registered/Active/Resolved` ; уточнив `Food`, `Neutral Defense recovery`, `City wealth growth`, `Transit candidates`, `attacker_destination_mode` для атаки з `Camp` і автоматичний вихід `Knight` із `Castle` при старті `Movement`.

Все залишив одним комітом:

`16d88571fc40bb64311fbbc5229a4a2428ef1fd5`

Є два моменти, які навмисно не чіпав, бо тут уже потрібне твоє рішення: звідки саме береться один спільний `defender_combat_threshold` , якщо в `defense` беруть участь кілька `Army` з різними `defense_loss_threshold` ; і чий `allow_transit` має бути `fallback` у `Occupied Region`, коли фактичний `defender` — `occupier`, а характеристика зараз належить `Region/formal owner`.

спільний `defender_combat_threshold` - це мінімум з усіх армій у `Occupied Region` немає `allow_transit` , транзит там не викликає бойової ситуації

Worked for 2m 42s

Вніс обидва рішення в MODELS.md .

Тепер `defender_combat_threshold` не задається окремо: на Battle Start він обчислюється як мінімальний `defense_loss_threshold` серед усіх `defender Army`, що фактично залишилися в battle після `retreat/join` рішень. Окремий `user_set_defender_combat_threshold()` прибраний.

Для `Occupied Region` зафіксував, що `Transit` не створює `CombatSituation` і `allow_transit` там не застосовується. Якщо `Transit`-ситуація була зареєстрована ще до `Occupation`, але до її `Start Region` стала `Occupied`, вона завершується без battle. `Camp-entry` в `Occupied Region` як і раніше може створити `CombatSituation` проти `occupier`.

Commit: 97a65b2a8da84ddde8185e422810c9fe9ba6d7db

проаналізуй ще раз

Worked for 13m 51s

Перевірив ще раз і вніс додаткові правки. Зокрема знайшов кілька неочевидних неузгодженостей: `Allow` помилково міг приводити до виконання вже вибраного `Retreat`; `queued Transit` перевіряв факт `Occupation` «колись до `Start`», а не стан саме на `Start`; після встановлення `Occupation` пізніша `Transit Army owner` все ще могла зареєструвати `CombatSituation`; не було механізму автоматичного скасування `Founding` при смерті/виході `founder`; не було чітко зафіксовано `eligibility Army`, що входить у `Camp` точно в момент `Start`. Також уточнив `lock` порогів, `Food` у `Movement`, `combat strengths`, `connectivity recalculation`, `Annexation vs active Founding`, `lifecycle Founding` і `Movement`.

Все залишив одним комітом:

11b5e118f9abf3ff2716cb3c0ecbcd94b411ea7

Залишилися два помітні питання, які вже потребують твого рішення, тому їх не чіпав: що саме відбувається з початковим `Movement/route Transit Army` після `user_join_defense_from_transit()` — вона після бою залишається в `Region` чи може продовжити старий `Transit`; і як поводить `Retreat` у порожню чужу `Owned Region` — це спеціальний `retreat` без звичайного `arrival/occupation interaction` чи після досягнення `Camp` запускаються звичайні правила чужої `Region`.

1) вважаємо, що перед боєм армії, що долучаються до оборони стають в кемп. Відповідно всі наслідки бою для них такі самі, як і для тих що вже були в кемпу. І, дорочче, у нас було правило, що якщо для армії що прогнала є декілька областей, що підходять для відступу - обирається випадково. Так от - якщо в захисті було

декілька армій, і вони програли- область відступу вибирається одна на всіх, а не окремо для кожної

Stopped thinking

2) Retreat викликає окупацію

Worked for 37s

Зафіксував обидва рішення.

Army, що долучається до defense з Transit, перед боєм переходить у Camp і надалі нічим не відрізняється від інших defender Army щодо результатів бою.

При поразці кількох Army однієї сторони вибирається одна спільна допустима Region для Retreat; якщо варіантів кілька — випадково. Усі Army відступають туди.

Retreat у чужу Owned Region після досягнення Camp викликає Occupation.

Commit: dfdafab1ffa7e5fa286d074c7960aea7518f6bf2

аналізуй

Worked for 8m 58s

Ще раз пройшовся по MODELS.md і виправив те, що не потребувало нових рішень: joined-from-Transit Army тепер реально завершує Movement і входить у Camp ; Retreat явно не запускає нову CombatSituation у retreat Region і може одразу привести до Occupation; уточнив спільний вибір retreat Region для сторони; виправив resource/Food/Gold/Silver flow для Occupied/disconnected Region; прибрав застарілу згадку abstract combat із retreat geometry.

Commit: e3eb038a9237aad8708fa10495b7ee81f4c3539f

Залишилися два питання, де потрібне твоє рішення.

Перше: якщо кілька defender Army програли, але **немає жодної Region, допустимої одночасно для всіх**, хоча кожна Army окремо має куди відступати. Раніше правило було: якщо Army не має допустимого Retreat — її threshold стає максимальним. Тепер із вимогою одного спільного напрямку треба визначити, чи в такому випадку весь Defender side отримує максимальний threshold.

Друге: explicit attack з Camp у Neutral Region. Запас attacker threshold визначається через target_opponent . Якщо Army прийшла саме в Neutral Region, її target_opponent за маршрутом зазвичай null , тому явна атака на B використовуватиме incidental_combat_threshold , хоча

Player вручну вибрав B як ціль атаки. Треба визначити, чи це правильно, чи така явна атака повинна використовувати `target_combat_threshold`.

що значить кожна Army окремо має куди відступати ? Правило одне для всіх армій

Worked for 1m 1s

Так. Моє формулювання про «кожна Army окремо має куди відступати» було неправильним.

У нас Retreat destination визначається на рівні всієї combat side. Для defender side з кількох Army є один набір допустимих Region і один вибір Region для всіх Army, що відступають. Якщо допустимих Region немає взагалі, спільний defender threshold для цього battle стає максимальним.

Це виправив у `MODELS.md`, прибрав per-Army retreat sets/intersection logic.

Commit: `ec5048d507600509025d518feab70ddf3ee6e363`

2. `target_opponent` - є в Movement, при атаці в нейтральній області він взагалі не до чого. Використовується основний поріг, це ж свідомо атака

Worked for 36s

Так. `target_opponent` має сенс лише в контексті Movement.

Для explicit attack з Camp у Neutral Region ціль уже явно вибрана гравцем, тому використовується `target_combat_threshold`; `target_opponent` у цій логіці взагалі не бере участі.

Виправив опис і Registration semantics.

Commit: `6a57e5cefe220c0c150eecea6dd803b8678d05c3`

по попередньому пункту уточнення. Я мав на увазі не те, що ми обмежуємо вибір області відступу тільки тими областями, куди можуть відступити всі армії, а те що в принципі не може бути такого, що одна армія може відступити в якусь область, а інша - ні. Чим вони відрізняються?

Так, тут ти правий. Я помилково переніс допустимість Retreat на рівень окремої Army.

У межах однієї сторони бою всі Army перебувають в одній Region, мають одного противника і одну роль у цьому бою. Усі критерії допустимості напрямку Retreat залежать саме від цього: напрямку

атаки/оборони та стану сусідньої Region, а не від характеристик конкретної Army. Тому ситуації «Army 1 може відступити в X, а Army 2 не може» взагалі не повинно існувати.

Отже, правильна модель — `legal_retreat_regions` визначається **один раз для combat side**. Якщо варіантів кілька, одна Region вибирається випадково; всі Army цієї сторони, які мають відступити, йдуть туди.

У попередньому коміті я вже частково перейшов на side-level, але залишив формулювання, що виглядають як наслідок «перетину можливостей Army». Це треба дочистити.

аналізуй

Worked for 2m 9s

Проаналізував саме наслідки цього уточнення й дочистив `MODELS.md`.

Знайшов ще одну реальну проблему: одного `retreat_region` для всієї `CombatSituation` недостатньо. Може бути, наприклад, pre-battle Retreat частини defender Army, а потім сам battle програє Attacker. Тоді потрібні одночасно destination для Defender side і окремий destination для Attacker side. Тому розділив на `defender_retreat_region` та `attacker_retreat_region`.

Також прибрав залишки per-Army логіки допустимості: `get_legal_retreat_regions(role)` тепер однозначно працює для всієї combat side; якщо side взагалі не має допустимого Retreat — її threshold стає максимальним. Pre-battle Retreat теж не може бути вибраний, якщо у defender side немає допустимого напрямку.

Commit: e69127c5906f27faeedd2fcda32d023e75640375 .

Є ще одне вже справжнє питання. У старому `01_GAME_RULES_v1.md` при виборі між кількома власними Region написано «ближча до власного Castle». Якщо в defense беруть участь Army, пов'язані з різними Castle, це суперечить щойно уточненій side-level логіці: тоді пріоритет може залежати від конкретної Army. Тут треба визначити один критерій для всієї Defender side.

пріоритет визначається головнокомандувачем армії

не так. Тут не власний замок армії, а власний замок області

Так, зрозумів поправку. Йдеться не про home Castle конкретної Army.

Для Defender при виборі між власними неокупованими Region пріоритет має Region, ближча до **Castle, до якого належить поточна combat Region**. Тобто критерій єдиний для всієї Defender side і від складу Army не залежить.

Це якраз повністю узгоджується із side-level Retreat.

виправляй та аналізуй

Worked for 2m 42s

Виправив і заодно звірив пов'язану Retreat-логіку.

Уточнення зафіксоване так: для власної кандидатної Region пріоритет визначається її відстанню до Castle, до якого приєднана саме ця Region. Це не home Castle Army, не Castle Commander і не Castle combat Region. Тому критерій лишається однаковим для всієї сторони.

Також у 01_GAME_RULES_V1.md прибрав застарілі формулювання, де Retreat визначався окремо для кожної Army, і синхронізував Target/Incidental Combat з уже прийнятою моделлю: explicit attack у Neutral Region використовує Target Combat Threshold незалежно від target_opponent .

Зміни зведені в один commit: ca72f116cafd51c194673ff152fa6464f41c5c77 .

Після цього в перевірній частині Retreat-логіки суперечностей не бачу.

перевір все

Worked for 25m 23s

Перевірив усі документи проєкту, не лише останній блок Retreat. Звірив 01_GAME_RULES_V1.md , MODELS.md , glossary, config format, Game Events, обидва update-файли, базову концепцію й backend architecture.

Автоматично виправив усе, що не потребує нового рішення: старі Food/City/Neutral Defense/Regrouping/CombatSituation правила в 01_GAME_RULES_V1 , переніс target_opponent з Army у Movement , синхронізував Dt , Occupied Transit, FIFO, side-level Retreat, explicit Neutral attack, прибрав RegionControl із event draft, додав відсутні user events, зробив Knight replacement наслідком будь-якого Knight.die() . Також знайшов ще один Retreat-баг: порожня чужа Castle Region формально проходила як допустима foreign Region — тепер вона явно заборонена. Два проміжні *_update.md позначив як інтегровані, щоб вони більше не були другим суперечливим джерелом правил. 00_BASE і backend architecture змін не потребували.

Commit: c3472931e17742958f6b583c0de05650ba41b3b9 . Це один commit від попереднього стану.

Залишилися питання, які вже не можна виправити без твого рішення:

1. Player задав pre-battle Retreat, але до Battle Start допустима Region зникла/стала недопустимою. Що робити з таким наказом?

2. Army почала Retreat у порожню чужу Owned Region, але за її Dt до Camp туди встигли прийти війська власника. Зараз правило «Retreat не створює нову CS і після Camp arrival робить Occupation» дає конфлікт.
3. Merge/Split дозволений під час Regrouping, але не визначено, який regrouping_progress отримує результат. Особливо якщо merge двох Army з різним progress.
4. Founder після створення нового Castle змінює home Castle. У старому Castle звільняється Palace slot — не визначено, чи він породжує нового Knight/replacement.
5. Найсерйозніший технічний пробіл: якщо Commander гине, Army.dissolve_after_commander_death() виконується під час casualties, але після цього ще треба застосувати результат battle — Retreat/Occupation/Transit continuation/Regrouping. Треба визначити, як створені після розпаду однозагінні Army успадковують thresholds і поточний post-combat state.

Я б наступним розібрав саме п. 5 — він впливає на коректність самої моделі combat.

Today 10:39 AM

1. pre-battle Retreat - локально для бойової ситуації підмінює поріг втрат на 0%
при розрахунку бою спочатку перевіряється можливість відступу для обох сторін, і якщо немає - поріг знімається на 100%
тільки після цього армії, в яких встановлено 0 поріг відступають
2. заборонене відступати в чужу обоасть, в яку уже ввійшли для кемпу якісь війська
3. не дозволений
4. так, той самий механізм як після загибелі
5. Вцілілий лицар з найбільшим досвідом автоматично призначається командувачем

Worked for 7m 5s

Вніс усі 5 уточнень і перевінив наслідки.

Ключове виправлення по Retreat: pre-battle Retreat тепер не окрема дія відступу, а локальна підміна effective threshold конкретної Army на 0% . На Battle Start спочатку для обох сторін визначається можливість Retreat; якщо її немає — threshold сторони стає 100% . Лише потім усі effective 0%

відступають. Для Defender після їх виходу спільний threshold береться як minimum серед тих, хто залишився.

Також:

- у чужу Owned Region не можна Retreat, якщо там уже є будь-яка Camp/Regrouping/entered-for-Camp Army;
- Merge/Split під час Regrouping заборонені;
- founder, переходячи в новий Castle, запускає KnightReplacement у старому Castle;
- смерть Commander більше не розпускає Army: після всіх casualties командиром стає вцілілий Knight з найбільшим Experience, Army зберігає свій стан і thresholds.

При однаковому Experience залишив детермінований технічний tie-breaker, щоб replay був однозначним.

Commit: cfaf0a433f66d76ba7eb27f8066535d6fba46cd6 .

You've reached the maximum length for this conversation, but you can keep talking by starting a new chat.